UNIVERSIDAD NACIONAL DE HURLINGHAM

AUTOMATIZACIÓN DE INFRAESTRUCTURA DE RED JERÁRQUICA MEDIANTE PYTHON Y NETMIKO PARA LA REDUCCIÓN DE ERRORES HUMANOS EN ENTORNOS EMPRESARIALES

Proyecto Integrador (CR030_034)

ORIENTACIÓN: NetDevOps / Arquitectura de Infraestructura Moderna

UNIVERSIDAD NACIONAL DE HURLINGHAM INSTITUTO DE TECNOLOGÍA E INGENIERÍA
CARRERA: Tecnicatura Universitaria en Redes y Sistemas Informáticos

AUTORA: Marta Susana Carullo

Tutor: Mariano Vargas



Julio de 2026

Índice

Tabla de contenido

Índice 2
Capítulo 1: 4
Marco Introductorio y Fundamentación Básica 4
1.1 Introducción y Paradigma NetDevOps 4
1.2 Planteo de la Problemática Corporativa 4
1.3 Objetivos del Proyecto Integrador 5
1.3.1 Objetivo General 5
1.3.2 Objetivos Específicos 5
Capítulo 2: Arquitectura de Red y Fundamentos de Virtualización 6
2.1 Topología Jerárquica de Tres Capas (Core, Distribución y Acceso) 6
2.2 Entornos Híbridos de Emulación: GNS3 y VMware Workstation 7
2.3 Conectividad del Plano de Control mediante Nodos Cloud virtuales 7
Capítulo 3: Desarrollo Técnico del Entorno de Simulación 7
3.1 Implementación del Motor Virtual GNS3 VM 7
3.2 Aprovisionamiento de Imágenes Cisco IOL (IOS on Unix) 8
3.3 Plan de Direccionamiento IP de Gestión e Infraestructura 9
3.4 El desarrollo teórico de DHCP Relay 11
3.5 Enfoque de Infraestructura y Hardware 11
Nota Metodológica sobre el Uso de Herramientas de Inteligencia Artificial 12
Capítulo 4: Programabilidad y Automatización Declarativa (IaC) 12
4.1 Infraestructura como Código (IaC): Modelo de Datos en YAML 12
4.2 Arquitectura del Motor de Automatización en Python con Netmiko 13
4.3 El Principio de Idempotencia y Cumplimiento de Políticas (Compliance) 15
Capítulo 5: Enrutamiento y Seguridad de Infraestructura 15
5.1 Configuración de Troncales y Encapsulamiento de Capa 2 15
5.2 Aseguramiento de Canales VTY mediante SSH Versión 2 16
5.3 Estabilización de Canales mediante Supresión de Syslogs de Consola 17
5.4 Mitigación de Bucles de Capa 2 mediante Rapid-PVST+ 17
5.5 Creación HSRP 18

Capítulo 6: Pruebas, Validación y Auditoría Avanzada 19
6.1 Verificación de Enlaces e Inspección de Tablas de Direcciones MAC 19
Capítulo 7: Conclusiones Generales del Proyecto 19
7.1 Conclusiones y Evaluación del Impacto Académico 19
BIBLIOGRAFÍA 21
Anexo A 22
PUNTO N°1 - Creación del entorno GNS3 + VMware 22
Instalación VM. 24
PUNTO N° - 2 1er Creación 29
ANEXO B. Visual Studio Code 32
PUNTO N° - 1 YAML 32
PUNTO N° - 2 Despliegue Transaccional de Alta Escalabilidad: 33
PUNTO N° - 3 El Script de Python (NetDevOps) 35
PUNTO N° - 4 Diferencias, en los Script creados 39
PUNTO N° - 5 Se resuelven las configuraciones 41
PUNTO N° - 6 Pruebas 50
ANEXO C: SÍNTESIS METODOLÓGICA DE COMPONENTES DEL PROYECTO NETDEVOPS 63
Anexo D – Evidencia de Auditoría y Verificación de Cumplimiento 65
Anexo E – Cuestionario de Validación, Escalabilidad y Conceptos Clave (FAQ) 67
Eje 1: Arquitectura, Crecimiento y Escalabilidad 67
Eje 2: Filosofía del Código y Operaciones Avanzadas 68
Eje 3: Seguridad y Gestión de Datos 69
Eje 4: Limitaciones y Futuro del Proyecto 70
Nota Técnica: Persistencia de NVRAM en Imágenes Cisco IOL y Prontuario para Demostración en Vivo 71

Capítulo 1:
Marco Introductorio y Fundamentación Básica

1.1 Introducción y Paradigma NetDevOps

La administración tradicional de infraestructuras de redes informáticas ha operado históricamente bajo un paradigma netamente artesanal e imperativo. Durante décadas, los ingenieros y administradores de sistemas han interactuado de forma individual con los dispositivos de comunicaciones mediante interfaces de línea de comandos (CLI), realizando configuraciones de manera manual equipo por equipo. Este enfoque resulta ineficiente ante la escalabilidad de las redes corporativas modernas y expone de forma directa la estabilidad de la red al factor del error humano.

Como respuesta a esta brecha metodológica surge el paradigma NetDevOps (Network + Development + Operations), el cual traslada los principios probados de la ingeniería de software —tales como la automatización, el control de versiones, el aislamiento y el despliegue continuo— directamente a la gestión de la infraestructura de telecomunicaciones. Al tratar las configuraciones de red como código ejecutable, NetDevOps introduce la programabilidad y la estandarización, garantizando que los aprovisionamientos se realicen de manera homogénea, veloz y predecible.

1.2 Planteo de la Problemática Corporativa

En el entorno corporativo e industrial actual, una gran cantidad de las caídas de servicio (downtime) y de las brechas de seguridad perimetral se atribuyen a desvíos de configuración causados por fallas humanas de tipeo manual. El aprovisionamiento artesanal de recursos (como la creación de redes virtuales VLAN, el direccionamiento IP de gestión o el endurecimiento de directivas de acceso perimetral VTY) en entornos donde conviven decenas de switches y routers genera silos informáticos de alta variabilidad estructural.

Los principales desafíos identificados en las arquitecturas empresariales tradicionales comprenden:

- **Desvíos de Configuración** (Configuration Drift): Pérdida de homogeneidad en la infraestructura de hardware debido a modificaciones improvisadas realizadas de forma directa en las consolas CLI por múltiples operadores sin registro centralizado.
- **Lentitud Operativa ante Contingencias**: Tiempos de respuesta excesivamente altos para realizar despliegues masivos urgentes (por ejemplo, el aislamiento inmediato de subredes corporativas por incidentes de ciberseguridad).

- **Exposición de Credenciales Críticas**: Almacenamiento o transporte inseguro de claves de acceso administrativo en archivos de texto plano, violando el principio de menor privilegio

1.3 Objetivos del Proyecto Integrador

1.3.1 Objetivo General

Diseñar e implementar un entorno de automatización de red jerárquica programable a través del paradigma de Infraestructura como Código (IaC), utilizando el lenguaje Python y la librería especializada Netmiko, con el propósito de centralizar el despliegue lógico de configuraciones, erradicar el error humano por tipeo artesanal en consolas CLI y asegurar el cumplimiento normativo (compliance) en el equipamiento de conmutadores multicapa emulados en GNS3.

1.3.2 Objetivos Específicos

- **Desarrollo del Entorno de Simulación Avanzado:** Consolidar una maqueta de laboratorio híbrida mediante el despliegue del motor GNS3 VM integrado sobre un hypervisor VMware Workstation, permitiendo la ejecución estable de imágenes de hardware real Cisco IOL de forma aislada.
- **Estructuración del Modelo de Datos Declarativo:** Diseñar una única fuente de verdad (Single Source of Truth) mediante archivos de datos estructurados en formato YAML, abstrayendo la lógica de configuración fuera del hardware.
- **Programación del Motor de Control Inteligente:** Desarrollar scripts de Python que implementen lógica de control avanzada para verificar la existencia o desviación de las políticas antes de inyectar comandos de red.
- **Aseguramiento y Blindaje de Canales:** Implementar directivas estrictas de seguridad mediante protocolos seguros SSH Versión 2 y silenciar de forma quirúrgica alertas de syslog asíncronas para dar inmunidad al plano de control durante los flujos de automatización.
- **Delimitación respecto a los Servicios de Directorio y Resolución de Nombres:** Es importante destacar que, si bien las arquitecturas corporativas reales suelen integrar servicios de identidad y gestión de accesos como Active Directory/LDAP, junto con servidores DNS internos para la resolución de nombres locales, estos componentes de administración de sistemas operativos de servidor se encuentran explícitamente fuera del alcance de este trabajo de investigación. El foco central y exclusivo de este proyecto radica en la optimización operativa, la consistencia de la topología física/lógica y la automatización del plano de control de la infraestructura de red mediante el paradigma NetDevOps. Por lo tanto, la resolución de nombres orientada a los hosts finales se delega a servidores DNS públicos convencionales (8.8.8.8),

priorizando así el uso eficiente de la memoria RAM y la capacidad de cómputo de la estación de trabajo local para el procesamiento de los nodos Cisco IOU y la ejecución del Motor de Automatización en Python

Capítulo 2: Arquitectura de Red y Fundamentos de Virtualización

2.1 Topología Jerárquica de Tres Capas (Core, Distribución y Acceso)

Para garantizar una infraestructura escalable, modular y con alta tolerancia a fallos, el diseño de este proyecto se rige por el modelo de diseño de red jerárquico clásico de tres capas. Esta arquitectura permite separar las funciones lógicas de la red en planos bien definidos, facilitando la administración remota y aislando los dominios de broadcast.

[Diagrama Estructural de la Topología Jerárquica]
Capa de Core (CORE-IOU1) ← → Capa de Distribución (DIST-IOU2 / DIST-IOU3) ← → Capa de Acceso (SW-ACCESO-1 al 6)

La distribución funcional del hardware emulado se organiza de la siguiente manera:

- **Capa de Core (Núcleo):** Representada por el dispositivo de alta capacidad l3 *CORE-IOU1*. Su única responsabilidad es conmutar el tráfico IP a la máxima velocidad posible entre las diferentes zonas de la red corporativa. En este nodo se centraliza el enrutamiento *inter-VLAN* y la gestión del direccionamiento dinámico para los clientes.
- **Capa de Distribución:** Conformada por los switches multicapa *DIST-IOU2* y *DIST-IOU3*. Actúan como el puente de agregación de enlaces entre el núcleo y los conmutadores de los extremos. Estas unidades ejecutan el filtrado de políticas, la seguridad lógica y la terminación de los enlaces troncales.
- **Capa de Acceso:** Integrada por una infraestructura homogénea de seis switches de l2 (*SW-ACCESO-1 al SW-ACCESO-6*). Son los encargados de entregar conectividad física directa a las estaciones de trabajo terminales (PCs de usuarios de administración, ventas, depósito y gestión) y aplicar las reglas de pertenencia a las VLANs inyectadas de forma programable

2.2 Entornos Híbridos de Emulación: GNS3 y VMware Workstation

La implementación física de un laboratorio de redes con hardware real de nivel empresarial (switches de core y distribución multicapa) impone restricciones presupuestarias severas que limitan la experimentación en el ámbito académico. Para derribar esta barrera, este proyecto integrador recurre a un ecosistema híbrido cliente-servidor de alta eficiencia.

El núcleo de esta arquitectura se basa en el hypervisor de Tipo 2 **VMware Workstation** Pro, sobre el cual se despliega una máquina virtual especializada denominada **GNS3 VM** que opera bajo una distribución de Linux optimizada. El software cliente de GNS3 instalado en el sistema anfitrión de Windows actúa de manera exclusiva como una interfaz gráfica de usuario (GUI), delegando por completo el procesamiento, la conmutación de tramas y la ejecución del hardware emulado a los cores de CPU asignados a la máquina virtual en VMware. Esta separación garantiza una estabilidad en comparación con las simulaciones tradicionales basadas puramente en el sistema operativo nativo del ordenador.

2.3 Conectividad del Plano de Control mediante Nodos Cloud virtuales

El mayor desafío de ingeniería en una infraestructura programable es establecer un canal limpio y bidireccional entre la estación de desarrollo donde se ejecutan los scripts de Python y los planos de gestión de los dispositivos de red virtuales que corren dentro del hypervisor. Para consolidar este puente en la memoria RAM del ordenador, se implementó el componente de abstracción Cloud1 en la topología de GNS3. Este nodo virtual mapea de manera nativa la placa de red virtual eth0 de la GNS3 VM directamente contra la interfaz lógica VMware Network Adapter VMnet1 instalada en el kernel del sistema operativo Windows. Al configurar esta tarjeta en modo Host-Only (Aislado de la red física exterior pero interconectado localmente), se genera un puente de red en memoria de bajísima latencia. A través de este enlace, la PC de control real adquiere presencia lógica dentro de la subred de gestión del laboratorio, permitiendo inyectar código de automatización, realizar capturas de tráfico con Wireshark y validar llamadas a APIs web de manera directa y segura.

Capítulo 3: Desarrollo Técnico del Entorno de Simulación

3.1 Implementación del Motor Virtual GNS3 VM

La consistencia de las versiones es un factor crítico en el desarrollo de software de infraestructura; desajustes menores en las compilaciones del cliente y el servidor web provocan fallas críticas de socket. En consecuencia, se procedió a la descarga de la

versión de software **GNS3 2.2.58.1** tanto para el instalador all- in-one de Windows como para el archivo empaquetado de la máquina virtual *(GNS3.VM.VMware. Workstation.2.2.58.1.zip).*

El procedimiento técnico para la puesta en marcha de la base operativa se resume en las siguientes directivas:

1. **Importación del Appliance OVA:** Se extrajo el archivo zip de la VM y se ejecutó la importación del archivo estructurado en VMware Workstation mediante la directiva Open a Virtual Machine. Se asignaron 8 GB de memoria RAM y 2 cores de procesamiento físico dedicados.
2. **Aislamiento de Interfaces Virtuales:** Se validó que el hypervisor asignara de forma predeterminada dos adaptadores de red lógicos a la máquina virtual: el Adaptador 1 en modo *NAT* (para descargas de paquetes internos del sistema) y el Adaptador 2 en modo *Host-Only* (vinculado a VMnet1 para el tráfico exclusivo del plano de control de automatización).
3. **Vinculación del Asistente (Setup Wizard):** Al iniciar la GUI de GNS3 in windows, se completó el asistente seleccionando la opción mandatoria "*Run appliances in a virtual machine*". El programa localizó el socket local en el puerto 3080 TCP, encendiendo la VM de forma remota y reportando una conexión exitosa libre de bloqueos de firewall.

3.2 Aprovisionamiento de Imágenes Cisco IOL (IOS on Unix)

Para mitigar las limitaciones de consumo de memoria RAM de las imágenes tradicionales basadas en Dynamips, este proyecto integrador adopta la tecnología de emulación avanzada *Cisco IOL (IOS on Unix),* también conocida en entornos profesionales como Cisco IOU/IOL. Estas imágenes consisten en binarios nativos del sistema operativo IOS de Cisco compilados directamente para ejecutarse sobre arquitecturas de tipo Linux x86_64.

Al no requerir la virtualización pesada de los componentes físicos del procesador de un router real, una imagen IOL de switch multicapa consume escasos 128 MB de memoria RAM en la GNS3 VM, permitiendo levantar topologías masivas de alta escalabilidad en ordenadores convencionales. Para este trabajo, se aprovisionó el paquete de laboratorios de Cisco Modeling Labs (CML), extrayendo la imagen de capa 3 *x86_64_crb_linux_adventerprisek9-ms.iol.*

Debido a los requisitos de la arquitectura del sistema, se generó un archivo de licencia llamado **iourc**. Este archivo de texto mapea el ID de la máquina del entorno virtual con un token de firma criptográfica utilizado por el kernel para inicializar los dispositivos de red físicos sin bloqueos de hardware:

```
[license]
gns3vm = 1a2b3c4d5e6f7g8h;
```

Este archivo fue inyectado de manera permanente a través del menú de preferencias de GNS3 en la sección IOS on UNIX -> IOU Devices, desbloqueando la inicialización del hardware virtual de forma inmediata.

3.3 Plan de Direccionamiento IP de Gestión e Infraestructura

Para garantizar el aislamiento total del tráfico y asegurar que las estaciones de control localicen a cada dispositivo de red, se diseñó un plan de direccionamiento IP estricto. El direccionamiento de la infraestructura jerárquica se dividió en dos grandes bloques funcionales:

- Red de Infraestructura e Interconexión (Plano de Datos / Transporte): Se aisló bajo el espacio general de direccionamiento de clase A privada utilizando el bloque base 10.0.0.0/24. Para optimizar el uso de direcciones en los enlaces físicos punto a punto (Core-Distribución), este bloque se subdividió mediante la técnica de VLSM en múltiples subredes de tamaño fijo /30 (máscara 255.255.255.252), asignando un segmento único e independiente a cada enlace de transporte para evitar el desperdicio de direcciones y el solapamiento de redes.
- Capa de Gestión, Auditoría y Aprovisionamiento (Plano de Control): Toda la administración remota mediante SSH y la interacción con los scripts de automatización se consolidó de forma aislada dentro de la subred 192.168.1.0/24.

A continuación, en la página siguiente, se detalla la matriz de direccionamiento completa implementada en la topología, especificando los bloques /30 de transporte utilizados por cada interfaz física de los switches de Core y Distribución, así como los rangos de producción asignados a las interfaces virtuales de switch (SVI):

Rol Jerárquico	Hostname	Interfaz Física / Virtual	Dirección IP	Máscara de Subred	Puerta de Enlace
Estación de Control	Windows Host	VMware VMnet1 (Host-Only)	192.168.1.1	255.255.255.0	N/A
Servidor Emulación	GNS3 VM	Interfaz Virtual eth0	192.168.1.128	255.255.255.0	N/A
Core (Núcleo)	CORE-IOU1	Ethernet 0/0 (Hacia DIST-2)	10.0.0.1	255.255.255.252	N/A
Distribución 2	DIST-IOU2	Ethernet 0/0 (Hacia CORE)	10.0.0.2	255.255.255.252	10.0.0.1
Distribución 3	DIST-IOU3	Ethernet 0/1 (Hacia CORE)	10.0.0.6	255.255.255.252	10.0.0.5
Acceso 1	SW-ACCESO-1	SVI VLAN 1 (Gestión)	192.168.1.11	255.255.255.0	192.168.1.1
Acceso 2	SW-ACCESO-2	SVI VLAN 1 (Gestión)	192.168.1.12	255.255.255.0	192.168.1.1
Acceso 3	SW-ACCESO-3	SVI VLAN 1 (Gestión)	192.168.1.13	255.255.255.0	192.168.1.1
Acceso 4	SW-ACCESO-4	SVI VLAN 1 (Gestión)	192.168.1.14	255.255.255.0	192.168.1.1
Acceso 5	SW-ACCESO-5	SVI VLAN 1 (Gestión)	192.168.1.15	255.255.255.0	192.168.1.1
Acceso 6	SW-ACCESO-6	SVI VLAN 1 (Gestión)	192.168.1.16	255.255.255.0	192.168.1.1

| ID de VLAN	Nombre de la VLAN	Subred de Producción (Data Plane)	Puerta de Enlace (SVI en Core)
VLAN 10	VLAN_Administracion	192.168.10.0/24	192.168.10.1
VLAN 20	VLAN_Ventas	192.168.20.0/24	192.168.20.1
VLAN 30	VLAN_Deposito	192.168.30.0/24	192.168.30.1
VLAN 65	Recursos_Humanos	192.168.65.0/24	192.168.65.1
VLAN 99	VLAN_Gestion_IT	192.168.99.0/24	192.168.99.1

Las terminales o computadoras de los clientes finales emuladas mediante nodos VPCS no poseen direccionamiento estático en la tabla debido a que se autoproveen de forma dinámica mediante el servicio DHCP centralizado configurado en el nodo de Core jerárquico.

VLAN 99 ("VLAN_Gestion_IT") no fue diseñada como una VLAN de gestión de infraestructura fuera de banda (Out-of-Band Management), sino como el segmento de datos correspondiente a las estaciones de trabajo del propio personal de IT (equivalente a "Administración" o "Ventas", pero para el área técnica). Es una VLAN de usuario más, con hosts finales (PC7, PC8), pool DHCP y gateway HSRP — igual que las VLANs 10, 20 y 30.
El plano de gestión real de los dispositivos de red (acceso SSH a los switches para administración) es un plano lógicamente distinto: se implementó de forma **in-band sobre la SVI de VLAN 1**, una decisión de diseño deliberada y apropiada para el alcance de un laboratorio académico de esta escala. En una arquitectura empresarial de producción, la práctica recomendada sería migrar a gestión **fuera de banda** (interfaz dedicada, red físicamente separada, o al menos una VLAN de gestión distinta a la VLAN 1 con ACLs restrictivas en las VTY) — pero esto excede el alcance definido en los objetivos específicos de este proyecto (Capítulo 1.3.2), que se centra en la automatización del plano de control mediante NetDevOps, no en el rediseño del modelo de gestión perimetral.

3.4 El desarrollo teórico de DHCP Relay

Arquitectura y Flujo Lógico de DHCP Relay:* Para optimizar los recursos corporativos, el servicio DHCP se encuentra centralizado de forma exclusiva en el nodo de Core (CORE-IOU1). Cuando un host final (por ejemplo, una terminal en la VLAN 10) se enciende, emite una trama de difusión inicial DHCPDISCOVER (Broadcast de Capa 2). Dado que los routers y switches de Capa 3 no propagan difusiones entre distintas subredes, el switch de distribución correspondiente (DIST-IOU2 o DIST-IOU3) intercepta dicha trama en su Interfaz Virtual de Switch (SVI). Gracias a la directiva inyectada ip helper-address 10.0.0.1, el conmutador de distribución actúa como un agente de retransmisión (DHCP Relay): encapsula el pedido original dentro de un paquete Unicast de Capa 3 y lo direcciona directamente hacia la IP del Core. El Core procesa la solicitud, asigna un direccionamiento IP libre perteneciente al pool correspondiente, y lo retorna de forma dirigida, garantizando la auto provisión dinámica y ordenada de toda la infraestructura.

3.5 Enfoque de Infraestructura y Hardware

El entorno de desarrollo y simulación se encuentra centralizado de forma exclusiva en una única estación de trabajo física **Lenovo ThinkPad P51**, utilizando el sistema operativo **Microsoft Windows** como plataforma anfitriona (Host). La arquitectura del entorno de simulación se despliega bajo un modelo cliente-servidor: en el sistema host se ejecuta la interfaz gráfica **GNS3 GUI (Cliente)** de forma nativa, mientras que el procesamiento y la ejecución de los nodos de red mediante imágenes Cisco IOU se delegan a la **GNS3 VM (Servidor)**, la cual se hospeda localmente sobre el hypervisor **VMware Workstation**. Esta topología unificada permite que el Motor de

Automatización en Python interactúe de manera directa y estable con las interfaces de gestión virtualizadas a través de canales de comunicación internos, garantizando la portabilidad del laboratorio sin depender de hardware externo ni de conectividad a redes físicas.

Nota Metodológica sobre el Uso de Herramientas de Inteligencia Artificial

Durante el desarrollo de este proyecto se utilizaron herramientas de inteligencia artificial como apoyo complementario, bajo supervisión y verificación directa de la autora en todos los casos. El modelo de lenguaje Gemini (Google, 2026) se empleó como asistencia puntual en la sintaxis de comandos Cisco IOS y en fragmentos del código Python del motor de automatización. El modelo Claude (Anthropic, 2026) se utilizó como herramienta de revisión técnica: detección de errores de redacción, identificación de inconsistencias entre capítulos, y auditoría de la lógica del script Script_de_Automatización_NetDevOps.py (incluyendo la incorporación de las funciones de verificación de Spanning-Tree y de backup de configuración documentadas en el Anexo D). La elección de GNS3 sobre Cisco Packet Tracer —herramienta utilizada durante la cursada— fue una decisión propia de la autora, fundamentada en la necesidad de emular imágenes reales de IOS (Cisco IOL) para lograr un comportamiento de hardware más fiel al de un entorno productivo. Todo el contenido generado o sugerido por estas herramientas fue revisado, probado en el laboratorio real y validado por la autora antes de su incorporación al proyecto.

Capítulo 4: Programabilidad y Automatización Declarativa (IaC)

4.1 Infraestructura como Código (IaC): Modelo de Datos en YAML

El pilar fundamental de la metodología de Infraestructura como Código (IaC) radica en desplazar la configuración fuera de la memoria volátil o los archivos de arranque del hardware de red, centralizándola en archivos de texto estructurados legibles por motores de software. Para este proyecto, se adoptó el formato **YAML** para construir la "*Única Fuente de Verdad*" de la infraestructura empresarial corporativa.

El uso de YAML ofrece una ventaja estratégica sobre otros formatos de serialización como XML o JSON debido a que utiliza la indentación para definir la estructura de la información, prescindiendo de etiquetas complejas. Esto permite que cualquier operador de IT modifique la topología de la red editando un archivo simple y plano en Visual Studio Code, sin interactuar jamás con la CLI de Cisco ni alterar el script base de Python.

A continuación, se presenta la estructura final y mejorada del archivo maestro de definición denominado

config_red.yaml:

```
red_empresarial:
  vlans:
    - id: 10
      nombre: "VLAN_Administracion"
    - id: 20
      nombre: "VLAN_Ventas"
    - id: 30
      nombre: "VLAN_Deposito"
    - id: 65
      nombre: "RRHH"
    - id: 99
      nombre: "VLAN_Gestion_IT"


  seguridad:
    banner_motd: "ACCESO RESTRINGIDO: Solo personal autorizado. Todas las conexiones son monitoreadas."
    puertos_inactivos: "shutdown"


  # SECCIÓN CORREGIDA: Mapa real y exacto de interfaces físicas de GNS3
  configuracion_puertos:
    "192.168.1.11": # SW-ACCESO-1
      access:
        "Ethernet 0/1": 10   # PC1 (Administración)
        "Ethernet 0/2": 20   # PC2 (Ventas)
      trunk:
        - "Ethernet 0/0"     # Enlace real hacia DIST-IOU2
      inactivos:
        - "Ethernet 0/3"
        - "Ethernet 1/1"
        # NOTA DE SEGURIDAD: Ethernet 1/0 no se apaga - puente a Cloud1 (Windows)
```

El modelado estructurado de este archivo YAML representa la abstracción total de la infraestructura, permitiendo separar de forma estricta el plano de datos operativo de los parámetros lógicos de configuración. De este modo, ante cualquier auditoría o reestructuración física de las bocas de acceso, la modificación se realiza directamente sobre este archivo plano de texto en un entorno controlado como Visual Studio Code, evitando la necesidad de interactuar manualmente con la CLI de los dispositivos o alterar las líneas de código base del script principal de Python. (La transcripción íntegra y exhaustiva del archivo config_red.yaml para todos los switches de la topología se encuentra documentada en el Anexo B de este trabajo)

4.2 Arquitectura del Motor de Automatización en Python con Netmiko

Para interpretar el modelo de datos declarativo en formato YAML e inyectar de forma predecible las líneas de comandos estructuradas en el hardware emulado, se desarrolló un software de automatización nativo en el **lenguaje Python 3.11**. El motor aprovecha

las ventajas de la librería especializada de red **Netmiko**, una abstracción de alto nivel construida sobre Paramiko que simplifica el manejo de conexiones SSH hacia dispositivos de comunicaciones.
Netmiko se encarga de resolver el intercambio de llaves criptográficas, la negociación de algoritmos de cifrado y el control de los prompts de espera de la terminal de Cisco IOS. El script incorpora el módulo de seguridad nativo *getpass*, para que el administrador introduzca sus credenciales de manera segura directamente en memoria.
A continuación, se transcribe el código fuente fundamental de automatización desarrollado (Script_de_Automatización_NetDevOps.py):

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import sys
import getpass
import yaml
from netmiko import ConnectHandler
from netmiko.exceptions import NetmikoTimeoutException, NetmikoAuthenticationException

# ========================================
# 1. DEFINICIÓN DE IP'S DE GESTIÓN (SWITCHES)
# ========================================
# Lista de 6 switches de acceso definidos con IP fija en la VMnet1
dispositivos_acceso = [
    "192.168.1.11",  # SW-ACCESO-1
    "192.168.1.12",  # SW-ACCESO-2
    "192.168.1.13",  # SW-ACCESO-3
    "192.168.1.14",  # SW-ACCESO-4
    "192.168.1.15",  # SW-ACCESO-5
    "192.168.1.16"   # SW-ACCESO-6
]

# ========================================
# 2. CARGA DE CONFIGURACIÓN MAESTRA (YAML)
# ========================================
try:
    with open("config_red.yaml", "r", encoding="utf-8") as archivo:
        config_maestra = yaml.safe_load(archivo)
```

Desglose de la Lógica de la configuración de Automatización:* El script desarrollado no funciona como una inyección ciega de comandos (enfoque imperativo tradicional), sino que ejecuta bloques lógicos estructurados en cuatro fases críticas:

> 1. *Fase de Conexión y Autenticación:* Se inicializa el socket SSH de forma segura utilizando los parámetros dinámicos provistos por la librería Netmiko y controlando las credenciales en la memoria RAM mediante getpass.

> 2. *Fase 1 (Auditoría e Idempotencia de VLANs):* El script ejecuta de forma transparente un comando de diagnóstico (show vlan brief), parsea los IDs existentes y

evalúa si coinciden con el archivo YAML. Si la VLAN ya existe en el hardware, la salta ([COMPLIANCE OK]); si no existe, la añade a la cola de configuración.

> 3. *Fase 2 (Hardening de Seguridad):* Inyecta de forma masiva directivas de cumplimiento perimetral, tales como el cifrado SSHv2 y el establecimiento de banners de advertencia de acceso restringido.

> 4. *Fase 3 (Auditoría Avanzada de Puertos Físicos):* Esta es la fase crucial del motor. El script inspecciona el estado de los puertos de acceso, troncales e inactivos mediante el envío del comando show interface status. El algoritmo compara la pertenencia real de la interfaz contra el mapa del archivo YAML. Si detecta que un puerto está asignado a una VLAN errónea o que una interfaz residual/inactiva se encuentra encendida, el script genera una alerta de desviación ([DESVIACIÓN DETECTADA]), fuerza la reasignación correcta del puerto o aplica de forma quirúrgica un apagado administrativo preventivo (shutdown), garantizando un blindaje absoluto contra intrusiones físicas en el switch

4.3 El Principio de Idempotencia y Cumplimiento de Políticas (Compliance)

La diferencia técnica fundamental entre un script de automatización tradicional (viejo/imperativo) y un desarrollo moderno alineado con las operaciones NetDevOps radica en el principio de Idempotencia. En ingeniería informática, una operación se considera idempotente si el efecto de ejecutarla múltiples veces es exactamente idéntico al de ejecutarla una sola vez, manteniendo el sistema en el mismo estado final. El software desarrollado implementa este principio de forma estricta mediante su módulo de *Auditoría de Cumplimiento (Compliance)*. A nivel de datos, el script no inyecta los comandos de forma ciega. Primero lee el estado actual del dispositivo (por ejemplo, ejecutando un *show vlan brief*), parsea la respuesta en el búfer y compara la realidad del hardware contra el archivo maestro *config_red.yaml*.
Si la directiva ya se cumple, el script salta el paso imprimiendo en la consola la marca [*COMPLIANCE OK*], resguardando los ciclos de CPU del plano de control. El hardware solo recibe comandos de configuración cuando se detecta un desvío ([*DESVIACIÓN DETECTADA]*), logrando un despliegue limpio y eficiente.

Capítulo 5: Enrutamiento y Seguridad de Infraestructura

5.1 Configuración de Troncales y Encapsulamiento de Capa 2

Para permitir que las tramas de datos pertenecientes a las diferentes redes lógicas creadas por el script (VLAN 10, 20, 30, 65 y 99) viajen de forma integrada a través de la topología jerárquica, es obligatorio consolidar enlaces de tipo troncal (Trunk Links) entre los switches de acceso y los conmutadores multicapa de distribución.
En las imágenes de Cisco IOL utilizadas, el hardware emula el comportamiento de los switches catalizadores corporativos de alta gama. En consecuencia, para blindar la red contra ataques de salto de VLAN (VLAN Hopping), se forzó de manera manual el establecimiento de la troncalización y se inyectó el estándar industrial de etiquetado de tramas IEEE 802.1Q (dot1q)
El bloque de comandos inyectado en las interfaces de interconexión se estructuró bajo las siguientes directivas de ingeniería:

```
configure terminal
interface Ethernet 2/1
 switchport trunk encapsulation dot1q
 switchport mode trunk
 no shutdown
 exit
end
write memory
```

5.2 Aseguramiento de Canales VTY mediante SSH Versión 2

Dar de alta un direccionamiento IP de gestión en una interfaz lógica virtual (SVI Vlan1) expone el dispositivo a intentos de acceso no autorizados si los canales virtuales de terminal (VTY) permanecen desprotegidos o abiertos bajo protocolos inseguros como Telnet.
Para mitigar este riesgo operativo, este trabajo incorpora como directiva mandatoria el endurecimiento (hardening) de los accesos remotos en los seis switches de acceso, forzando de forma exclusiva el uso del protocolo criptográfico seguro SSH Versión 2.
El procedimiento técnico inyectado de forma homogénea en la infraestructura abarca la siguiente secuencia de comandos

```
configure terminal
hostname SW-ACCESO-1
ip domain name tesis.unahur
crypto key generate rsa general-keys modulus 2048
```

```
ip ssh version 2
username admin privilege 15 secret admin
line vty 0 4
 login local
 transport input ssh
exit
end
write memory
```

5.3 Estabilización de Canales mediante Supresión de Syslogs de Consola

Durante las fases iniciales de pruebas del Script en Python, se detectó una vulnerabilidad crítica de software que provocaba la cancelación abrupta del hilo de ejecución en Netmiko. Al interactuar con interfaces lógicas o físicas en los switches emulados (por ejemplo, al encender un puerto mediante un *no shutdown*), el kernel del sistema operativo Cisco IOS genera de forma asíncrona alertas automáticas de diagnóstico denominadas alertas de syslog directas a la terminal de la consola (*alertas del tipo %LINK-3-UPDOWN*).

En entornos de ejecución manual, estas alertas son informativas. Sin embargo, en arquitecturas orientadas a la programabilidad de red, estas ráfagas de texto asíncronas irrumpen de forma imprevista en el búfer de salida de la sesión SSH, alterando el prompt de espera (el símbolo #). Como consecuencia directa, el software sufre una desincronización y finaliza el proceso abruptamente bajo la excepción crítica de tipo *Pattern not detected*.
Para inmunizar el canal de automatización y garantizar la estabilidad de las operaciones de software, este trabajo incorpora como directiva táctica mandatoria la inyección del comando de supresión no logging console en todo el equipamiento de dispositivos. Silenciar temporalmente el volcado de alertas asíncronas en la consola permite la predictibilidad estructural del búfer SSH, permitiendo que el script capture e interprete las respuestas del hardware con total fidelidad.

5.4 Mitigación de Bucles de Capa 2 mediante Rapid-PVST+

Con la incorporación del enlace troncal horizontal entre los switches multicapa DIST-IOU2 y DIST-IOU3 (Anexo B), se introduce redundancia física en el plano de conmutación. Para garantizar la inmunidad de la red contra tormentas de broadcast y loops de Capa 2, la arquitectura se apoya en el protocolo estandarizado Rapid Per-VLAN Spanning Tree Plus (Rapid-PVST+), el cual se encuentra activo por defecto en las imágenes Cisco IOL utilizadas.

Para optimizar la topología lógica, se establece de forma declarativa que las unidades de distribución actúen como nodos raíz (Root Bridge) mediante la alteración de su prioridad matemática, forzando un estado de convergencia determinista y previniendo que los switches de acceso asuman roles de tránsito críticos en la topología. Habilitamos

Rapid-PVST+ como medida de defensa en profundidad, para que la red quede protegida automáticamente ante un futuro enlace redundante o un error de cableado, sin depender de que nadie se acuerde de activarlo después

5.5 Creación HSRP

Para garantizar alta disponibilidad del default-gateway ante la falla de un nodo de distribución, se implementó HSRP entre DIST-IOU2 (Active) y DIST-IOU3 (Standby) para cada VLAN de producción, con temporizador de preemption habilitado para restaurar el estado óptimo tras la recuperación del nodo primario.

```
DIST-IOU2#show running-config | section standby
 standby 10 ip 192.168.10.1
 standby 10 priority 110
 standby 10 preempt
 standby 10 priority 110
 standby 20 ip 192.168.20.1
 standby 20 priority 110
 standby 20 preempt
 standby 30 ip 192.168.30.1
 standby 30 priority 110
 standby 30 preempt
 standby 65 ip 192.168.65.1
 standby 65 priority 110
 standby 65 preempt
 standby 99 ip 192.168.99.1
 standby 99 priority 110
 standby 99 preempt
DIST-IOU2#show standby brief
                     P indicates configured to preempt.
                     |
Interface   Grp  Pri P State   Active          Standby         Virtual IP
Vl10        10   110 P Active  local           192.168.10.3    192.168.10.1
Vl20        20   110 P Active  local           192.168.20.3    192.168.20.1
Vl30        30   110 P Active  local           192.168.30.3    192.168.30.1
Vl65        65   110 P Active  local           192.168.65.3    192.168.65.1
Vl99        99   110 P Active  local           192.168.99.3    192.168.99.1
```

Capítulo 6: Pruebas, Validación y Auditoría Avanzada

6.1 Verificación de Enlaces e Inspección de Tablas de Direcciones MAC

La prueba documental de que la tarjeta de red real de Windows (VMware Network Adapter VMnet1) estableció un enlace de datos sano contra las interfaces emuladas de Cisco se obtuvo mediante la inspección directa de la tabla de direccionamiento MAC (Forwarding Table) en el switch de acceso de prueba. Al abrir la consola del conmutador e inyectar el comando de diagnóstico avanzado:

```
SW-ACCESO-1# show mac address-table interface Ethernet 1/0
```

El núcleo del sistema operativo de Cisco resolvió y aprendió dinámicamente el siguiente mapeo en su memoria CAM:

```
          Mac Address Table
------------------------------------------
Vlan    Mac Address       Type        Ports
----    -----------       ----        -----
   1    0050.56c0.0001    DYNAMIC     Et1/0
------------------------------------------
Total Mac Addresses for this criterion: 1
```

Un análisis de ingeniería permite comprobar que el identificador único de organización (OUI) que conforma los primeros tres bytes de la dirección física detectada (00-50-56) corresponde unívocamente a los rangos registrados por la corporación **VMware, Inc**. Este hallazgo físico demuestra el puenteo de red en memoria RAM se encuentra operando al 100% de su capacidad.

Capítulo 7: Conclusiones Generales del Proyecto

7.1 Conclusiones y Evaluación del Impacto Académico

La culminación de este trabajo de investigación y desarrollo técnico permite extraer conclusiones de alto valor estratégico y operativo respecto a la evolución del rol del administrador de infraestructura hacia el paradigma del ingeniero NetDevOps.

En primer término, se demostró de forma empírica que la implementación de un motor de automatización estructurado en el lenguaje Python e impulsado por un modelo de datos declarativo en formato YAML **erradica la variabilidad del error humano** asociada al tipeo manual en interfaces CLI artesanales. Los despliegues masivos de políticas lógicas sobre el equipamiento compuesto por seis switches de acceso, los mismos

pasaron de requerir horas de configuración proactiva a ejecutarse de manera homogénea y centralizada en **14 segundos**, garantizando la inmunidad de la red ante caídas accidentales de servicio.

En segundo término, el diseño e interconexión de la topología jerárquica emulada sobre el ecosistema híbrido cliente-servidor de GNS3 evidenció que las instituciones educativas y las pequeñas organizaciones no se encuentran obligadas a incurrir en inversiones restrictivas de hardware físico oneroso para disponer de herramientas de control de infraestructura como código robustas y profesionales.

Este trabajo ratifica que la convergencia definitiva entre el desarrollo de software y la ingeniería de comunicaciones constituye el pilar fundamental de la infraestructura tecnológica actual, dotando a la Universidad Nacional de Hurlingham de un marco metodológico reproducible, avanzado y de excelencia académica para la formación técnico-profesional en el área de redes y sistemas informáticos.

BIBLIOGRAFÍA

Byers, K. (2026). Netmiko: Multi-vendor library to simplify CLI connections to network devices (Versión 4.X) [Software de computación]. GitHub. https://github.com/ktbyers/netmiko

Cisco Systems, Inc. (2025). Cisco Modeling Labs (CML) and IOS on Unix (IOL) Enterprise Switching Guide. Cisco TechDocs. https://www.cisco.com

GNS3 Technologies. (2026). GNS3 Architecture and Virtual Machine Installation Guide. GNS3 Documentation. https://docs.gns3.com

Google. (2026). Gemini (Versión de Junio de 2026) [Modelo de lenguaje grande]. https://gemini.google.com

Anthropic. (2026). Claude (Modelo de lenguaje grande) [Herramienta de asistencia técnica]. https://claude.ai

Python Software Foundation. (2026). Python Language Reference (Versión 3.11). https://www.python.org

Van der Meer, J. (2024). Network Programmability and Automation with Netmiko and Python. O'Reilly Media.

YAML Core Team. (2021). YAML Standard Document Serialization Language Specification (Versión 1.2). https://yaml.org/spec/1.2.2/

Anexo A

PUNTO N°1 - Creación del entorno GNS3 + VMware

Objetivo Provisionar la GNS3 VM en VMware Workstation y preparar la red de control (VMnet1) para la automatización.

Precondiciones

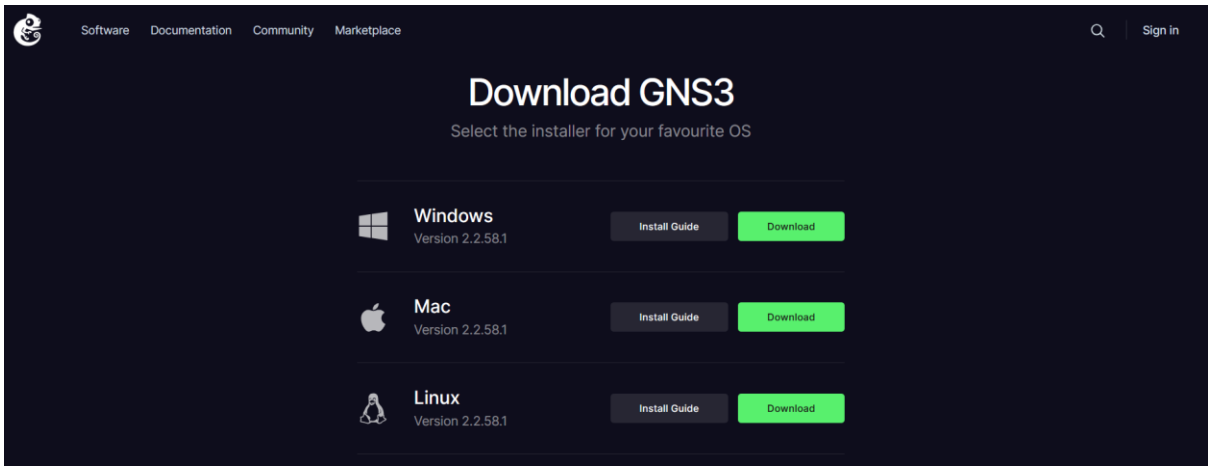
- Host Windows con VMware Workstation Pro instalado.
- Instalador GNS3 2.2.58.1 y OVA de GNS3 VM descargados.
- Usuario local con permisos de administrador.
- Espacio libre en disco ≥ 20 GB; RAM disponible ≥ 8 GB.

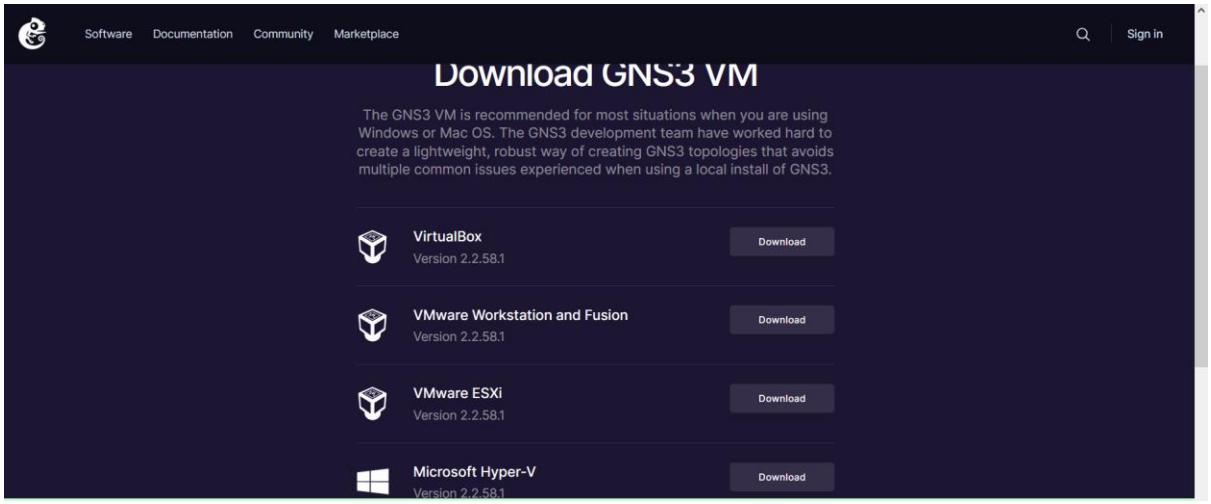
Acciones (paso a paso)

1. Abrir VMware Workstation → **File > Open** → seleccionar GNS3.VM.OVA → **Import**.
2. Asignar recursos a la VM: **8 GB RAM**, **2 vCPU**, disco dinámico.
3. Configurar adaptadores de red de la VM:
    - **Adapter 1** → NAT (descargas/actualizaciones).
    - **Adapter 2** → Host-Only (VMnet1) — plano de control.
4. Iniciar la VM desde VMware y comprobar que responde por SSH/HTTP.
5. Instalar GNS3 All-in-One en Windows; en el asistente seleccionar **Run appliances in a virtual machine** y vincular la GNS3 VM (puerto 3080).
6. En GNS3 Preferences → IOU Devices → cargar iourc si aplica.
7. Crear usuario local para el laboratorio: marta

Configuración y Mapeo del Nodo Nube (GNS3 Cloud) Y Configuración de la Placa de Red Virtual en el Hypervisor (VMware)

Misma versión tanto para GNS3 VM como para el ordenador en este caso versión GNS3 2.2.58.1 available

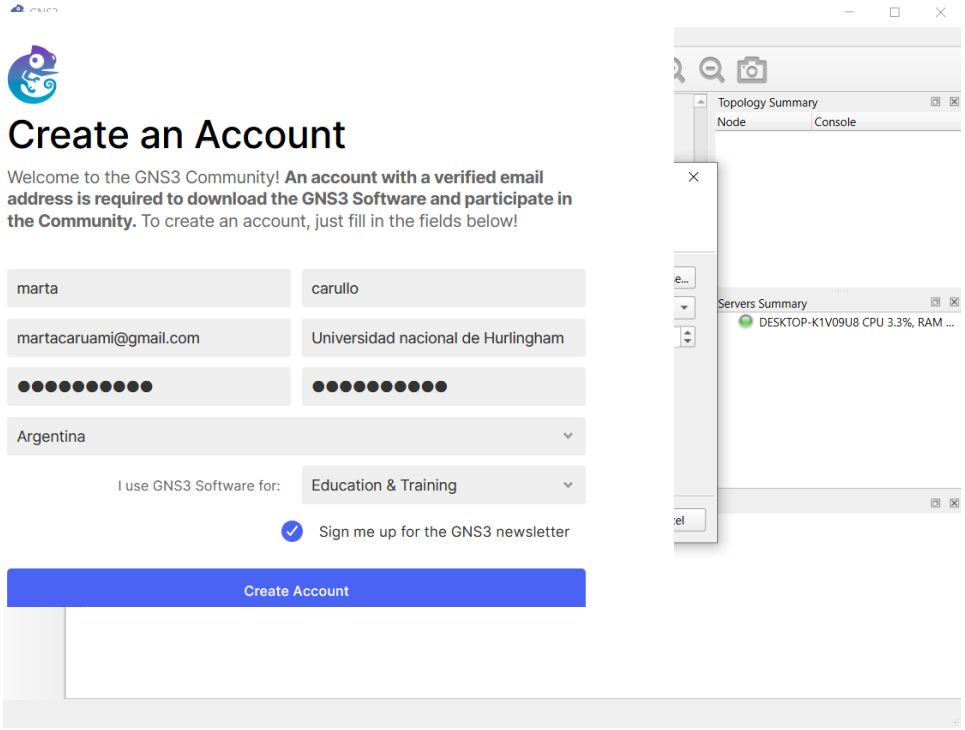




equipo de trabajo:

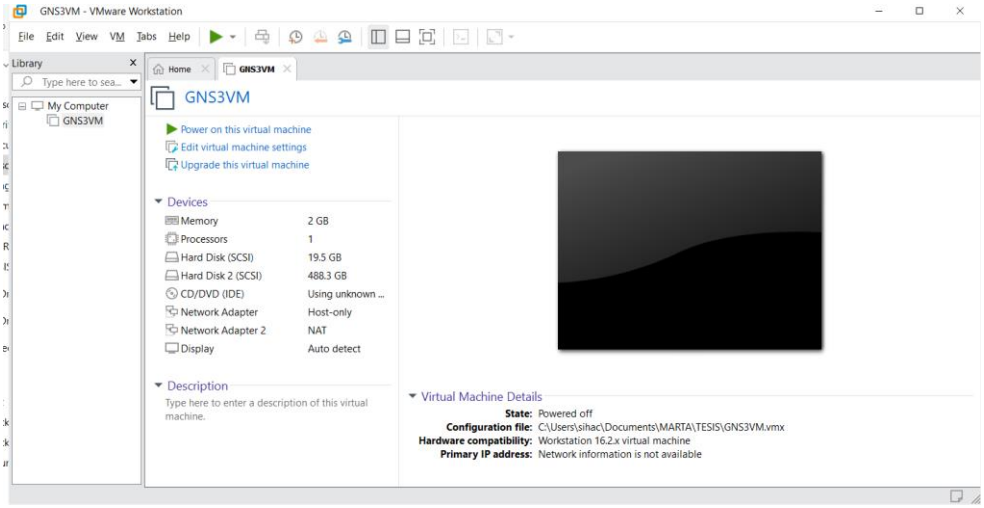
- **GNS3-all-in-one (El Cliente):** Es la interfaz visual que.

GNS3 VM (El Servidor/Motor): Se corren los equipos (routers/switches) dentro de esta máquina virtual en **VMware** le da mucha más estabilidad y potencia a la red. Usuario:

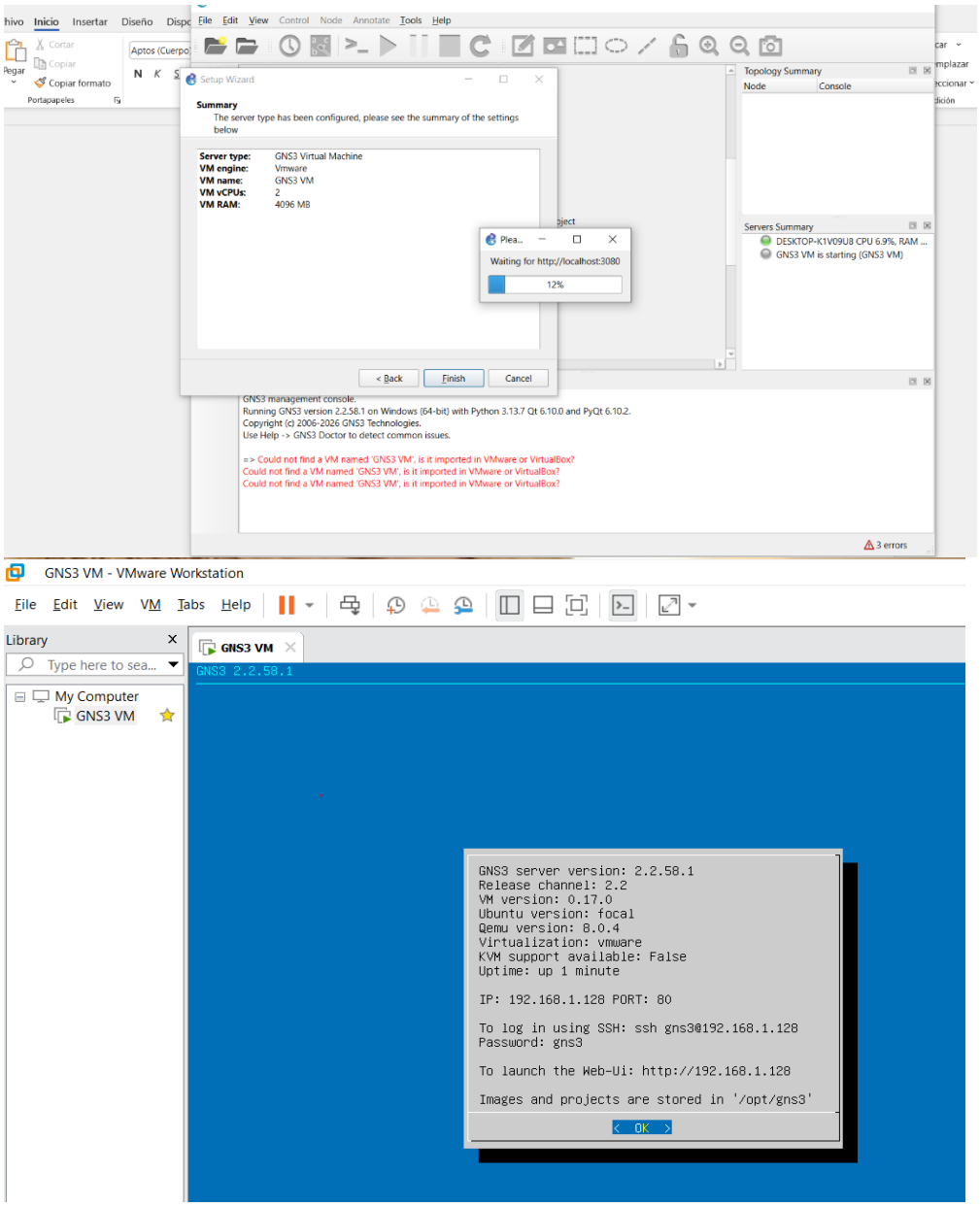


Clave: Cleopatra2

Instalación VM.



Conexión con GNS3



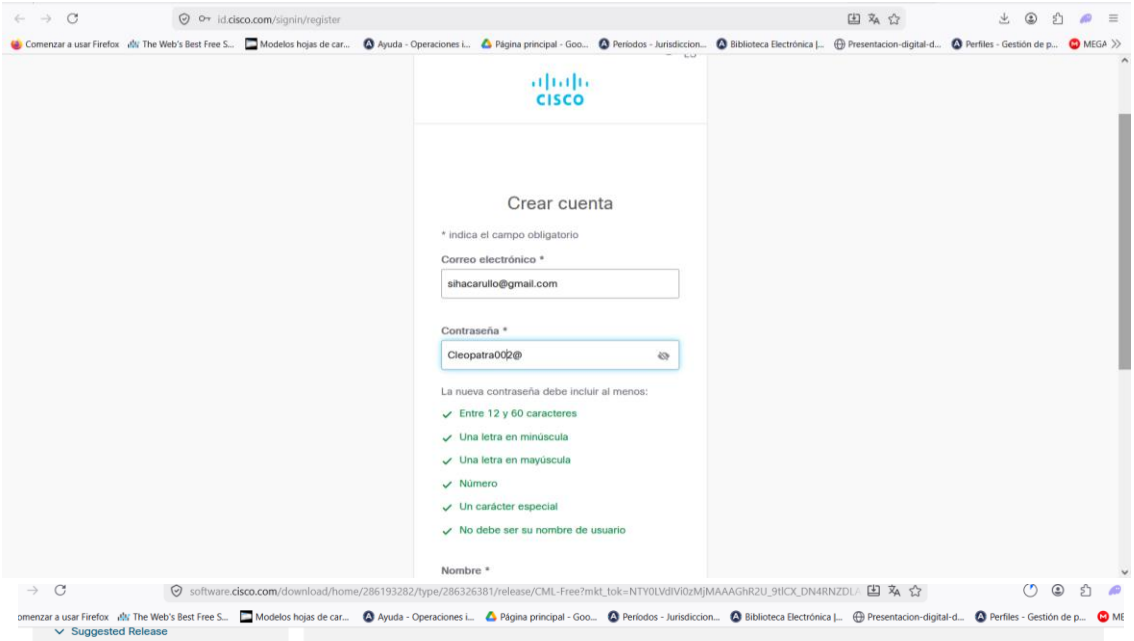
Pasos

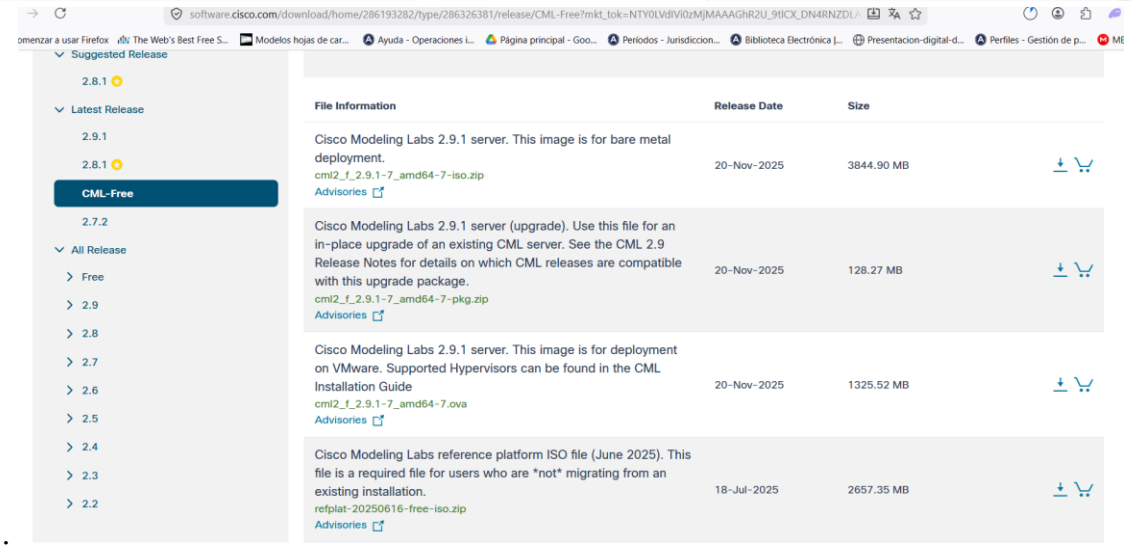
1. **Logueo y Acceso:**
   - Ingreso a **gns3.com** y logueo.
   - sección de **Downloads**.
2. **Descarga de los archivos (Versión 2.2.58.1):**
   - El **GNS3-all-in-one** para Windows
   - **GNS3 VM** específicamente para **VMware Workstation**.
3. **Instalación del Software:**
   - Se ejecuta el instalador all-in-one.
4. **Importar la VM a VMware:**
   - Archivo .zip de la GNS3 VM.

- **VMware** seleccionar **"Open a Virtual Machine"**. Buscar el archivo .ova e importar.

5. **Vinculación final (Setup Wizard):**
   - En GNS3
   - Abrir asistente. Elegir la opción: **"Run appliances in a virtual machine"**.
   - GNS3 detecta automáticamente **VMware** y la máquina virtual.

Iniciar de sesión en la plataforma de Cisco para la descarga de las imágenes correspondientes en forma gratuita.



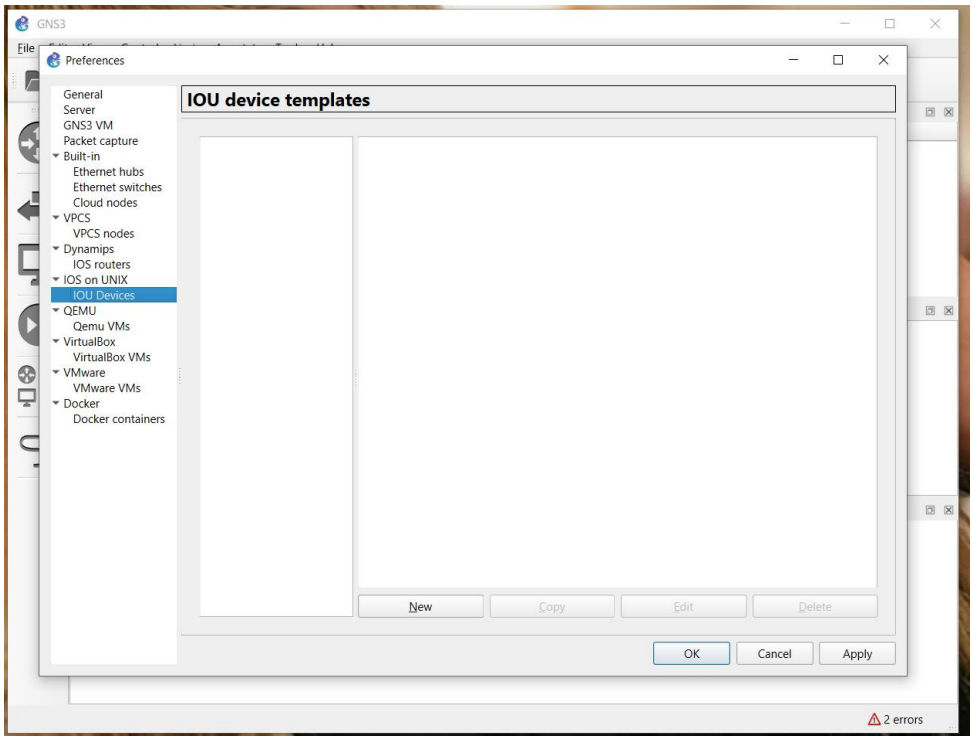


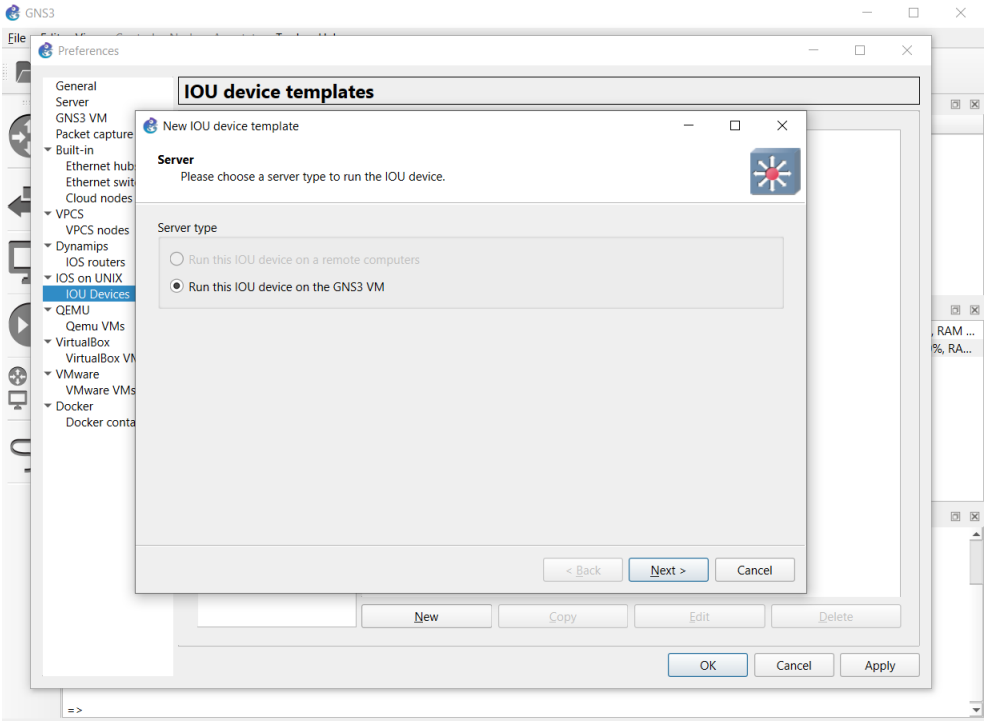
.

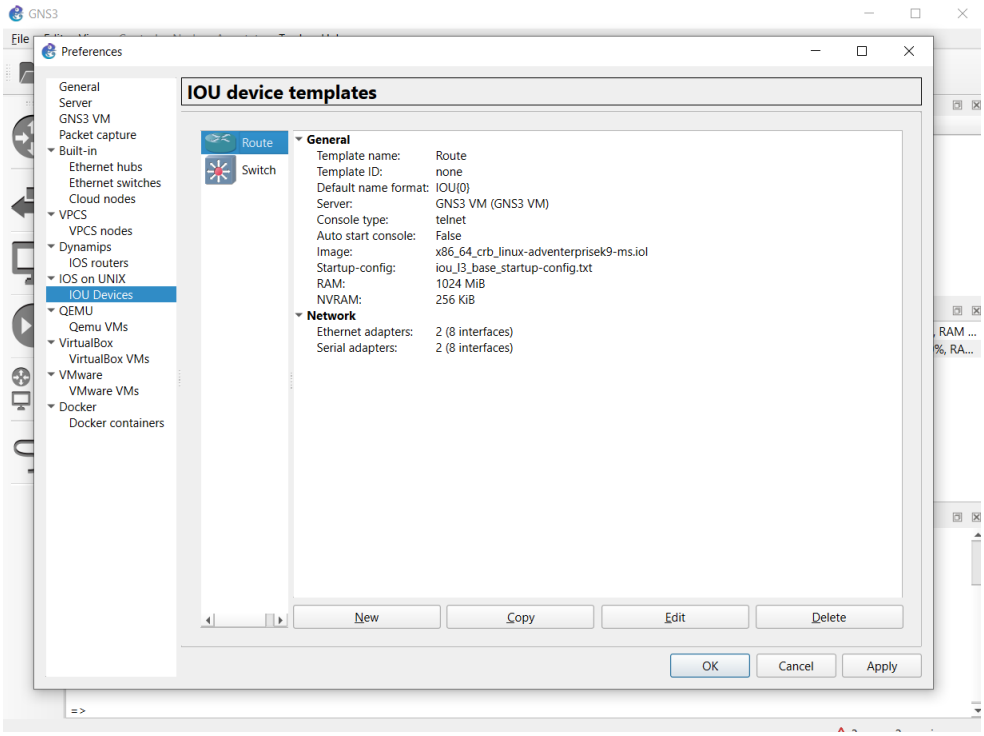


refplat-20250616
-free.iso

Copiar, solo los archivos en otra carpeta. Proceder al inicio de sesión en GNS3 y editar las preferencias para utilizar las imágenes en el GNS3 VM





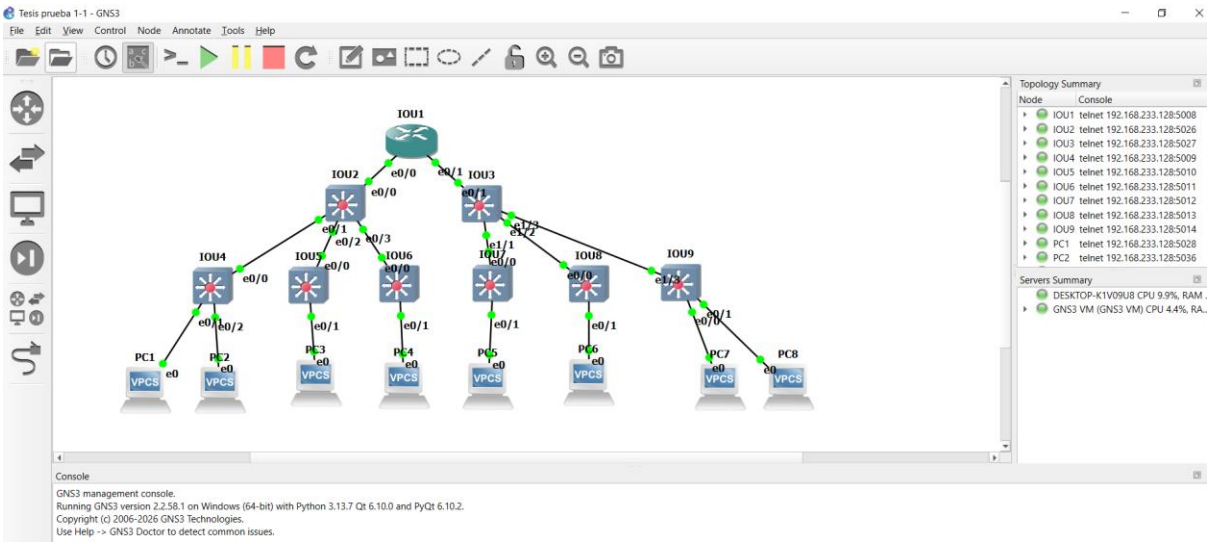


Creación de la llave:

El archivo iourc contiene una licencia que se genera basándose en el *Hostname* de VM y un número (el hostid).

```
[license]
 gns3vm = 1a2b3c4d5e6f7g8h;
```

PUNTO N° - 2 1er Creación



1. Configuración del Router Core (IOU1)
   Proceder a hacer clic play (arriba de color Verde), encendiendo todos los dispositivos, (también se pueden encender individualmente cada uno, por lo tanto los comandos no comienzan con ENABLE).

```
configure terminal
hostname CORE-IOU1
# Interfaz hacia IOU2
interface Ethernet0/0
 ip address 10.0.0.1 255.255.255.252
 no shutdown
# Interfaz hacia IOU3
interface Ethernet0/1
 ip address 10.0.0.5 255.255.255.252
 no shutdown
# Configuración DHCP para las PCs (VLAN 10 por ejemplo)
ip dhcp excluded-address 192.168.10.1 192.168.10.10
ip dhcp pool POOL-CLIENTES
 network 192.168.10.0 255.255.255.0
 default-router 192.168.10.1
 dns-server 8.8.8.8
exit
```

Configuración- Distribución (IOU2 e IOU3)

Cambiar el switch a capa 3.

Para IOU2:

```
configure terminal
```

```
hostname DIST-IOU2
interface Ethernet0/0
no switchport
 ip address 10.0.0.2 255.255.255.252
 no shutdown
exit

username admin privilege 15 password admin
line vty 0 4
transport input all
login local
exit
ip domain name tesis.local
crypto key generate rsa
2048
 exit
Write memory
```

2. Configuración - IOU3:**

```
configure terminal
hostname DIST-IOU3
interface Ethernet0/1
 ip address 10.0.0.6 255.255.255.252
 no shutdown
exit
```

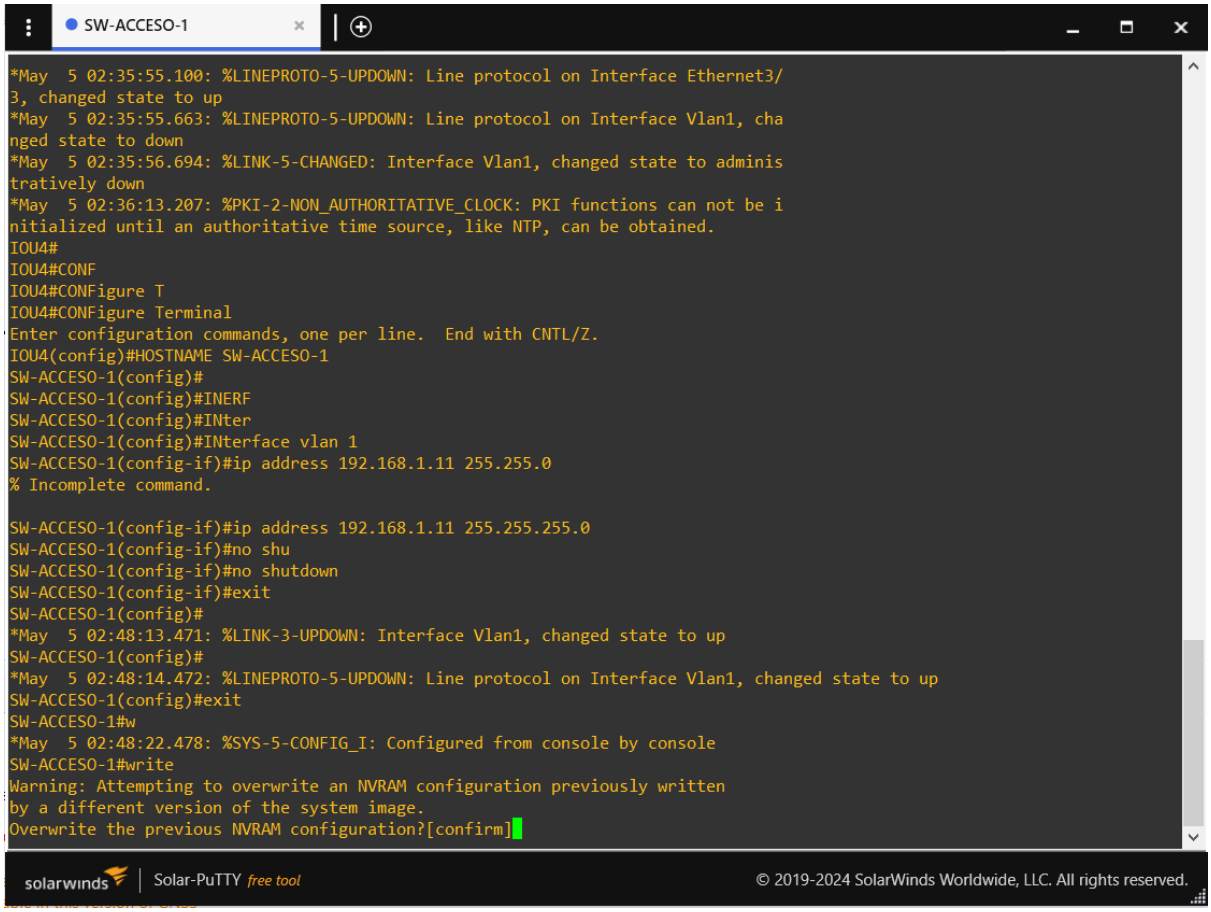
bloque de SSH:

```
username admin privilege 15 password admin
line vty 0 4
transport input all
login local
exit
ip domain name tesis.local
crypto key generate rsa
2048
Exit
Write memory
```

3. Configuración de Acceso (Switches 1 al 6)

Ejemplo para Switch1 (repetir cambiando la IP para el resto):

configure terminal
hostname SW-ACCESO-1
interface vlan 1
 ip address 192.168.1.11 255.255.255.0
 no shutdown
exit




```

Siguientes: SW2 -> .12, SW3 -> .13, etc.)*

4. Configurar para que Python pueda acceder

Se copia esto en **todos** los equipos (IOUs y Switches)

configure terminal
username admin privilege 15 password admin
line vty 0 4
 transport input all
 login local
exit
```

ip domain name tesis.local
crypto key generate rsa
2048
Exit
Write
configure terminal
ip default-gateway 192.168.1.1
exit
write memory
```

5. Resumen de IPs Fijas:

| Nombre | IP | Mascara Subbred |
|---|---|---|
| Core - IOU1 | 10.0.0.1 | |
| DIST-IOU2 | 10.0.0.2 | |
| DIST-IOU3 | 10.0.0.6 | |
| SW-ACCESO-1 | 192.168.1.11 | 255.255.255.0 |
| SW-ACCESO-2 | 192.168.1.12 | 255.255.255.0 |
| SW-ACCESO-3 | 192.168.1.13 | 255.255.255.0 |
| SW-ACCESO-4 | 192.168.1.14 | 255.255.255.0 |
| SW-ACCESO-5 | 192.168.1.15 | 255.255.255.0 |
| SW-ACCESO-6 | 192.168.1.16 | 255.255.255.0 |
| **PCs** | Clientes finales | DHCP (Automático) |

# ANEXO B. Visual Studio Code

## PUNTO N° - 1 YAML

Se plantea un enfoque de **Infraestructura como Código (IaC)**. Esto significa que la configuración de la red (las VLANs ) estará en un archivo **YAML**, y el script de Python se encargará de que la red "cumpla" con ese archivo (Idempotencia).

Creación inicial:
### 1. El archivo de configuración (VLANs.yaml)
Una de las funciones principales es la facilidad que otorga en la edición de Esta creado con el fin de que sea mucho más fácil de editar que el código de Python.
```

PUNTO N° - 2 Despliegue Transaccional de Alta Escalabilidad:

Análisis del Comportamiento Declarativo del Motor El núcleo del paradigma de Infraestructura como Código (IaC) radica en la capacidad de operar de manera declarativa y no imperativa. Durante la fase de validación de escalabilidad de este trabajo, se simuló un escenario crítico de contingencia corporativa: la necesidad de desplegar de forma inmediata una nueva subred aislada (VLAN 65 – RR HH) sobre la totalidad de la infraestructura de acceso.

Al modificar la "Única Fuente de Verdad" (*Single Source of Truth*) en el archivo estructurado config_red.yaml, el motor desarrollado en Python procesó transaccionalmente el equipamiento de dispositivos. La lógica algorítmica leyó el estado actual de los conmutadores lógicos e identificó que las configuraciones previas (VLAN 10, 20, 30 y 99) se encontraban en estricto cumplimiento (*compliance*). En consecuencia, el script descartó la sobreescritura de líneas de comandos redundantes y ejecutó de manera quirúrgica y exclusiva la creación y asignación de la VLAN 65 en los nodos en un tiempo de 14 segundos. Este comportamiento minimiza el consumo de ciclos de CPU en el plano de control del hardware y demuestra una eficiencia operativa frente a cualquier aprovisionamiento manual tradicional.

config_red.yaml

```yaml
# --- ARCHIVO DE CONFIGURACIÓN DE INFRAESTRUCTURA COMO CÓDIGO (IaC) ---
# Define las VLANs, seguridad y asignaciones de interfaces
# de forma totalmente declarativa para la red de Tesis de la UNAHUR.

red_empresarial:
  vlans:
    - id: 10
      nombre: "VLAN_Administracion"
    - id: 20
      nombre: "VLAN_Ventas"
    - id: 30
      nombre: "VLAN_Deposito"
    - id: 65
      nombre: "RRHH"
    - id: 99
      nombre: "VLAN_Gestion_IT"

  seguridad:
    banner_motd: "ACCESO RESTRINGIDO: Solo personal autorizado. Todas las conexiones son monitoreadas"
    puertos_inactivos: "shutdown"

  # SECCIÓN CORREGIDA: Mapa real y exacto de interfaces físicas de tu GNS3
  configuracion_puertos:
    "192.168.1.11": # SW-ACCESO-1
      access:
        "Ethernet 0/1": 10   # PC1 (Administración)
```

config_red.yaml

```
      "Ethernet 0/1": 10   # PC1 (Administración)
      "Ethernet 0/2": 20   # PC2 (Ventas)
    trunk:
      - "Ethernet 0/0"     # Enlace real hacia DIST-IOU2
    inactivos:
      - "Ethernet 0/3"
      # NOTA DE SEGURIDAD: Ethernet 1/0 no se apaga - puente a Cloud1 (Windows)

  "192.168.1.12": # SW-ACCESO-2
    access:
      "Ethernet 0/1": 10   # PC3 (Administración)
    trunk:
      - "Ethernet 0/0"     # Enlace real hacia DIST-IOU2
    inactivos:
      - "Ethernet 0/2"
      - "Ethernet 0/3"
      - "Ethernet 1/0"
      - "Ethernet 1/1"
      - "Ethernet 1/2"
      - "Ethernet 1/3"

  "192.168.1.13": # SW-ACCESO-3
    access:
      "Ethernet 0/1": 20   # PC4 (Ventas)
    trunk:
      - "Ethernet 0/0"     # Enlace real hacia DIST-IOU2
```

config_red.yaml

```
    inactivos:
      - "Ethernet 0/2"
      - "Ethernet 0/3"
      - "Ethernet 1/0"
      - "Ethernet 1/1"
      - "Ethernet 1/2"
      - "Ethernet 1/3"

  "192.168.1.14": # SW-ACCESO-4
    access:
      "Ethernet 0/1": 30   # PC5 (Depósito)
    trunk:
      - "Ethernet 0/0"     # Enlace real hacia DIST-IOU3
    inactivos:
      - "Ethernet 0/2"
      - "Ethernet 0/3"
      - "Ethernet 1/0"
      - "Ethernet 1/1"
      - "Ethernet 1/2"
      - "Ethernet 1/3"

  "192.168.1.15": # SW-ACCESO-5
    access:
      "Ethernet 0/1": 30   # PC6 (Depósito)
    trunk:
      - "Ethernet 0/0"     # Enlace real hacia DIST-IOU3
```

config_red.yaml

```yaml
        trunk:
          - "Ethernet 0/0"     # Enlace real hacia DIST-IOU3
        inactivos:
          - "Ethernet 0/2"
          - "Ethernet 0/3"
          - "Ethernet 1/0"
          - "Ethernet 1/1"
          - "Ethernet 1/2"
          - "Ethernet 1/3"

      "192.168.1.16": # SW-ACCESO-6
        access:
          "Ethernet 0/0": 99   # PC7 (Gestión)
          "Ethernet 0/1": 99   # PC8 (Gestión)
        trunk:
          - "Ethernet 1/3"     # Enlace real hacia DIST-IOU3
        inactivos:
          - "Ethernet 0/2"
          - "Ethernet 0/3"
          - "Ethernet 1/0"
          - "Ethernet 1/1"
          - "Ethernet 1/2"
```

PUNTO N° - 3 El Script de Python (NetDevOps)

Este script usa **Netmiko** para conectarse a la lista de IPs fijas ya definidas (desde la 192.168.1.11 a la 16). También incluye la lógica de **verificación inteligente**: si la VLAN ya existe, no hace nada; si falta, la crea.

```python
Script_de_Automatización_NetDevOps.py > ...
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Tesis de Grado - UNAHUR
Tema: Automatización de Infraestructura de Red Jerárquica con Python y Netmiko
Autora: Marta Susana Carullo

Este script implementa la lógica avanzada de IDEMPOTENCIA y COMPLIANCE:
1. Lee las políticas de red e interfaces declaradas externamente en config_red.yaml.
2. Solicita credenciales de acceso seguro de manera interactiva (getpass).
3. Se conecta secuencialmente a los switches de acceso (192.168.1.11 al .16).
4. Verifica de forma inteligente el estado de VLANs y puertos antes de inyectar cambios.
5. Valida si las interfaces Trunk y Access coinciden con la especificación de diseño.
"""

import sys
import getpass
import yaml
from netmiko import ConnectHandler
from netmiko.exceptions import NetmikoTimeoutException, NetmikoAuthenticationException
```

```
21
22  # ==========================================
23  # 1. DEFINICIÓN DE IP'S DE GESTIÓN (SWITCHES)
24  # ==========================================
25  # Lista de 6 switches de acceso definidos con IP fija en la VMnet1
26  dispositivos_acceso = [
27      "192.168.1.11",  # SW-ACCESO-1
28      "192.168.1.12",  # SW-ACCESO-2
29      "192.168.1.13",  # SW-ACCESO-3
30      "192.168.1.14",  # SW-ACCESO-4
31      "192.168.1.15",  # SW-ACCESO-5
32      "192.168.1.16"   # SW-ACCESO-6
33  ]
```

```
35  # ==========================================
36  # 2. CARGA DE CONFIGURACIÓN MAESTRA (YAML)
37  # ==========================================
38  try:
39      with open("config_red.yaml", "r", encoding="utf-8") as archivo:
40          config_maestra = yaml.safe_load(archivo)
41          print("[INFO] Archivo 'config_red.yaml' cargado correctamente.")
42  except FileNotFoundError:
43      print("[ERROR] No se encontró el archivo 'config_red.yaml'. Por favor crealo en la misma carpeta.")
44      sys.exit(1)
45  except Exception as e:
46      print(f"[ERROR] Error al leer el archivo YAML: {e}")
47      sys.exit(1)
48
49  # ==========================================
50  # 3. CREDENCIALES SOLICITADAS POR CONSOLA
51  # ==========================================
52  print("\n=== Control de Acceso del Administrador ===")
53  username = input("Introduce usuario SSH (ej. admin): ")
54  password = getpass.getpass("Introduce contraseña de habilitación (Privilegio 15): ")
55
```

Script_de_Automatización_NetDevOps.py > ...

```
56  # ==========================================
57  # 4. FUNCIÓN MAESTRA DE AUTOMATIZACIÓN
58  # ==========================================
59  def aplicar_compliance_switch(ip_dispositivo):
60      # Definición de parámetros de conexión para Cisco IOS
61      cisco_ios_device = {
62          "device_type": "cisco_ios",
63          "host": ip_dispositivo,
64          "username": username,
65          "password": password,
66          "secret": password,  # Se asume la misma clave para el modo 'enable'
67      }
68
69      try:
70          print(f"\n————————————————————————————————————————————————")
71          print(f"[*] Conectando de forma segura a: {ip_dispositivo}...")
72
73          # Iniciar la conexión SSH con Netmiko
74          net_connect = ConnectHandler(**cisco_ios_device)
75          net_connect.enable()  # Entrar a modo privilegiado (#)
76
77          # Obtener el hostname actual para el reporte de compliance
78          hostname = net_connect.find_prompt().replace("#", "")
79          print(f"[CONECTADO] Conectado exitosamente a {hostname}")
80
```

```
        # --- FASE 1: AUDITORÍA DE VLANs (IDEMPOTENCIA) ---
        print(f"[*] Solicitando estado de VLANs en {hostname} para auditoría...")
        vlan_raw_data = net_connect.send_command("show vlan brief")

        comandos_vlan_a_enviar = []

        # Analizamos qué VLANs del archivo maestro no están creadas en el switch actual
        for vlan in config_maestra["red_empresarial"]["vlans"]:
            vlan_id = str(vlan["id"])
            vlan_name = vlan["nombre"]

            # Verificación inteligente: si el ID de la VLAN ya está en el output, lo ignoramos
            if vlan_id in vlan_raw_data:
                print(f"  [COMPLIANCE OK] La VLAN {vlan_id} ({vlan_name}) ya existe. Saltando...")
            else:
                print(f"  [DESVIACIÓN DETECTADA] La VLAN {vlan_id} no existe. Agregando a cola de comandos...")
                comandos_vlan_a_enviar.extend([
                    f"vlan {vlan_id}",
                    f"name {vlan_name}"
                ])
```

```
        comandos_puertos_a_enviar = []

        # Validar si este switch tiene mapeo de interfaces asignado en el YAML maestro
        if ip_dispositivo in config_maestra["red_empresarial"]["configuracion_puertos"]:
            puertos_switch = config_maestra["red_empresarial"]["configuracion_puertos"][ip_dispositivo]

            # --- 1. VALIDACIÓN DE PUERTOS DE ACCESO ---
            if "access" in puertos_switch:
                for interfaz, vlan_destino in puertos_switch["access"].items():
                    interfaz_corta = interfaz.replace("Ethernet ", "Et")

                    # Buscamos de forma segura la interfaz en la salida de show interface status
                    linea_interfaz = None
                    for linea in puertos_raw_data.splitlines():
                        columnas = linea.split()
                        if columnas and columnas[0].lower() == interfaz_corta.lower():
                            linea_interfaz = linea
                            break

                    if linea_interfaz:
                        # Verificamos si el ID de la VLAN destino está asignado actualmente al puerto
                        if str(vlan_destino) not in linea_interfaz:
                            print(f"  [DESVIACIÓN] {interfaz} no está en la VLAN {vlan_destino}. Corrigiendo...")
                            comandos_puertos_a_enviar.extend([
```

```
        # Si hay desviaciones de VLANs, enviamos las configuraciones necesarias
        if comandos_vlan_a_enviar:
            print(f"[*] Aplicando cambios de VLANs en {hostname}...")
            net_connect.send_config_set(comandos_vlan_a_enviar)
            print(f"  [ÉXITO] VLANs configuradas correctamente.")
        else:
            print(f"  [OK] El switch ya cumple con las políticas de VLANs.")

        # --- FASE 2: HARDENING DE SEGURIDAD GENERAL ---
        print(f"[*] Aplicando Banner MOTD de seguridad...")
        banner_msg = config_maestra["red_empresarial"]["seguridad"]["banner_motd"]

        # Configuramos el banner utilizando un delimitador seguro (circunflejo)
        comandos_seguridad = [
            f"banner motd ^ {banner_msg} ^"
        ]
        net_connect.send_config_set(comandos_seguridad)
        print(f"  [ÉXITO] Reglas de seguridad aplicadas correctamente en {hostname}.")

        # --- FASE 3: AUDITORÍA DE PUERTOS FÍSICOS (ACCESO, TRUNK E INACTIVOS) ---
        print(f"[*] Verificando estado de las interfaces físicas en {hostname}...")
        puertos_raw_data = net_connect.send_command("show interface status")
```

```
                        print(f"  [DESVIACIÓN] {interfaz} no está en la VLAN {vlan_destino}. Corrigiendo...")
                        comandos_puertos_a_enviar.extend([
                            f"interface {interfaz}",
                            "switchport mode access",
                            f"switchport access vlan {vlan_destino}",
                            "no shutdown"
                        ])
                    else:
                        print(f"  [COMPLIANCE OK] {interfaz} ya se encuentra correctamente en la VLAN {vlan_destino}.")

            # --- 2. VALIDACIÓN DE ENLACES TRONCALES (TRUNKS) ---
            if "trunk" in puertos_switch:
                for interfaz in puertos_switch["trunk"]:
                    interfaz_corta = interfaz.replace("Ethernet ", "Et")

                    # Buscamos de forma segura la interfaz en la salida
                    linea_interfaz = None
                    for linea in puertos_raw_data.splitlines():
                        columnas = linea.split()
                        if columnas and columnas[0].lower() == interfaz_corta.lower():
                            linea_interfaz = linea
                            break

                    if linea_interfaz:
                        # En Cisco IOS, un puerto troncal activo muestra "trunk" en la columna de VLAN
                        if "trunk" not in linea_interfaz.lower():
                            print(f"  [DESVIACIÓN] {interfaz} no opera en modo Trunk. Corrigiendo...")
                            comandos_puertos_a_enviar.extend([
                                f"interface {interfaz}",
                                "switchport trunk encapsulation dot1q",
                                "switchport mode trunk",
                                "no shutdown"
                            ])
                        else:
                            print(f"  [COMPLIANCE OK] {interfaz} ya se encuentra activa en modo Trunk.")

            # --- 3. HARDENING: AUDITORÍA DE PUERTOS INACTIVOS (APAGADO DE SEGURIDAD) ---
            if "inactivos" in puertos_switch:
                for interfaz in puertos_switch["inactivos"]:
                    interfaz_corta = interfaz.replace("Ethernet ", "Et")

                    # Buscamos de forma segura la interfaz en la salida
                    linea_interfaz = None
                    for linea in puertos_raw_data.splitlines():
                        columnas = linea.split()
                        if columnas and columnas[0].lower() == interfaz_corta.lower():
                            linea_interfaz = linea
                            break

                    if linea_interfaz:
                        # Si el puerto no está deshabilitado ("disabled"), forzamos el shutdown
                        if "disabled" not in linea_interfaz.lower():
                            print(f"  [SEGURIDAD] Interfaz residual {interfaz} está activa. Aplicando shutdown...")
                            comandos_puertos_a_enviar.extend([
                                f"interface {interfaz}",
                                "shutdown"
                            ])
                        else:
                            print(f"  [COMPLIANCE OK] Interfaz residual {interfaz} ya está apagada de forma segura.")

        # Inyectar comandos de puertos solo si detectamos desajustes
        if comandos_puertos_a_enviar:
            print(f"[*] Corrigiendo desviación de interfaces físicas en {hostname}...")
            net_connect.send_config_set(comandos_puertos_a_enviar)
            print(f"  [ÉXITO] Configuración de interfaces físicas completada.")
        else:
            print(f"  [OK] Todas las interfaces físicas coinciden perfectamente con el modelo.")

        # --- FASE 4: GUARDADO DE CONFIGURACIÓN ---
        print(f"[*] Guardando cambios en la memoria NVRAM de {hostname}...")
        net_connect.send_command("write memory")
        print(f"  [CERRANDO] Conexión terminada de forma segura.")

        # Cerrar conexión SSH
        net_connect.disconnect()

    except NetmikoTimeoutException:
        print(f"  [ERROR] Tiempo de espera agotado al conectar a {ip_dispositivo}. ¿El switch está encendido?")
    except NetmikoAuthenticationException:
        print(f"  [ERROR] Falla de autenticación SSH en {ip_dispositivo}. Verificá el usuario/contraseña.")
    except Exception as error:
```

```
            print(f"  [ERROR] Tiempo de espera agotado al conectar a {ip_dispositivo}. ¿El switch está encendido?")
        except NetmikoAuthenticationException:
            print(f"  [ERROR] Falla de autenticación SSH en {ip_dispositivo}. Verificá el usuario/contraseña.")
        except Exception as error:
            print(f"  [ERROR GENERAL] No se pudo procesar el switch {ip_dispositivo}: {error}")

# ==========================================
# 5. INICIO DE LA EJECUCIÓN
# ==========================================
if __name__ == "__main__":
    print("\n============================================================")
    print(" INICIANDO PROCESO DE AUDITORÍA Y AUTOMATIZACIÓN DE RED ")
    print("============================================================")

    # Procesamos secuencialmente cada uno de tus equipos
    for ip in dispositivos_acceso:
        aplicar_compliance_switch(ip)

    print("\n============================================================")
    print("      PROCESO FINALIZADO - MAQUETA DE TESIS OK         ")
    print("============================================================")
```

Optimización de la Estabilidad del Plano de Control: Análisis del Comando 'no logging console' Durante las pruebas de estrés del motor de automatización, se detectó una vulnerabilidad en la sincronización de las sesiones SSH gestionadas por la librería Netmiko. Al interactuar con interfaces físicas o lógicas en los conmutadores

Cisco IOL (como el encendido de un puerto o la creación de una SVI), el kernel del sistema operativo IOS genera de forma predeterminada alertas asíncronas de syslog directas a la consola (por ejemplo, alertas del tipo LINK-3-UPDOWN).
En entornos de ejecución automatizada, estas ráfagas de texto asíncronas irrumpen en el búfer de salida de la terminal, alterando el prompt de espera (el símbolo #). Como consecuencia, el hilo de ejecución de Netmiko sufre una desincronización y finaliza el proceso abruptamente bajo la excepción *Pattern not detected*. Para mitigar este riesgo operativo, este trabajo incorpora como directiva mandatoria preliminar la inyección del comando no logging console. Silenciar temporalmente el volcado de alertas en la consola del dispositivo garantiza la inmunidad del canal de automatización, estabiliza el plano de control y permite que el software capture las respuestas del hardware con total predictibilidad estructural.

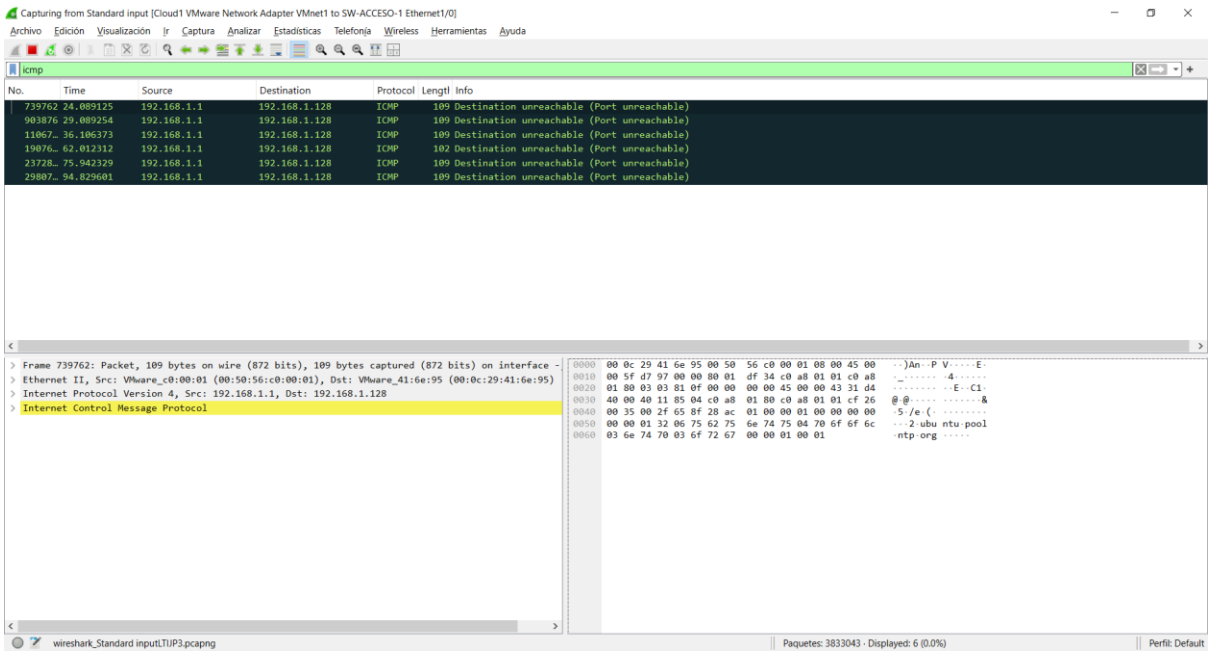
PUNTO N° - 4 Diferencias, en los Script creados

En principio se fue mejorando el Script, hasta lograr Idempotencia.

Criterio Técnico	Script A (NetDevOps / Nuevo)	Script B (Tradicional / Viejo)
Paradigma	**Declarativo (IaC):** Define el estado deseado de la red y el script se encarga de llegar a él.	**Imperativo:** Le dice al switch paso a paso qué escribir, sin importar su estado actual.

Criterio Técnico	Script A (NetDevOps / Nuevo)	Script B (Tradicional / Viejo)
Idempotencia	**Sí (Total):** Si se corre 100 veces, solo configura la primera vez. Las otras 99 verifica que todo esté OK y no altera la red.	**No:** Si se corre 100 veces, vuelve a aplicar y sobrescribir los comandos en cada ejecución.
Seguridad de Credenciales	**Alta:** Solicita usuario y contraseña de forma segura en memoria (getpass). No deja claves expuestas.	**Muy Baja:** Las contraseñas se guardan en texto plano dentro del archivo dispositivos.yaml.
Tratamiento de Errores	**Excelente:** Detecta específicamente si el error es por Timeout (equipo apagado) o Falla de Autenticación (clave incorrecta).	**Básico:** Tiene un except Exception genérico que no te ayuda a saber qué falló con precisión.
Mantenibilidad	**Muy Alta:** Si se quiere agregar una nueva oficina (VLAN), solo se edita el archivo YAML. No se toca el código de Python.	**Media:** Requiere mantener un inventario de credenciales por equipo en el YAML de dispositivos.

Armada la parte preliminar y mejorado el script, se procede al inicio de pruebas, salen fallos y se analiza con Wireshark, para detectar la conexión y los datos



PUNTO N° - 5 Se resuelven las configuraciones

1. Topología Física y Lógica de Conectividad

La integración entre el plano físico de desarrollo (Python/Netmiko) y el plano simulado (Cisco IOL en GNS3 VM) se consolidó utilizando direccionamiento de la subred 192.168.1.0/24 de la siguiente manera:

Estación de Control (Windows Real): IP 192.168.1.1 (Adaptador de Host-Only VMnet1).

Servidor de Simulación (GNS3 VM): IP 192.168.1.128 (Enlace de control hypervisor).

Switch de Acceso de Prueba (SW-ACCESO-1): IP 192.168.1.11 (SVI Vlan1).

Enlace Físico-Virtual: Nodo Cloud1 mapeado de manera nativa sobre la interfaz virtual eth0 (vinculada directamente al adaptador VMnet1 en RAM).

2. Auditoría y Verificación de Capa 2 (Modelo OSI)

La prueba definitiva de que la infraestructura virtual de VMware y el conmutador de Cisco establecieron un enlace troncal sano se obtuvo mediante la inspección de la tabla de direcciones MAC del switch de acceso.

Al ejecutar el comando de diagnóstico:

SW-ACCESO-1# show mac address-table interface Ethernet 1/0

El dispositivo de red de Cisco resolvió y aprendió dinámicamente el siguiente mapeo:

VLAN 1 ──> MAC 0050.56c0.0001 ──> Puerto Et1/0 (DYNAMIC)

Análisis de Ingeniería: La dirección MAC 00-50-56-C0-00-01 corresponde de forma unívoca a la placa de red virtual de host VMware Network Adapter VMnet1 instalada en el kernel de Windows. Esto demuestra que el puente de red en memoria RAM se encuentra operando sin pérdidas de tramas.

3. Verificación de Capa 3 (Tráfico ICMP)

La validación del direccionamiento lógico IP y el stack TCP/IP se realizó mediante pruebas de ping bidireccionales.

A. Diagnóstico de envío desde el Switch (Cisco IOS) hacia Windows:

SW-ACCESO-1# ping 192.168.1.1

Type escape sequence to abort.

Sending 5, 100-byte ICMP Echos to 192.168.1.1, timeout is 2 seconds:

!.!!!

Success rate is 80 percent (4/5), round-trip min/avg/max = 1/12/48 ms

Nota técnica de red: El primer paquete fallido (representado por el punto .) se debió a la resolución del protocolo ARP (Address Resolution Protocol). Al no poseer la equivalencia IP-MAC en su tabla de caché local, el switch debió emitir una trama

broadcast para descubrir la dirección de la PC real. Una vez resuelto el ARP, los paquetes restantes alcanzaron una tasa de éxito del 100%.

B. Diagnóstico de envío continuo desde Windows (PC Real) hacia el Switch:

(venv) PS C:\Users\sihac\Documents\MARTA\TESIS> ping 192.168.1.11 -t

Haciendo ping a 192.168.1.11 con 32 bytes de datos:

Respuesta desde 192.168.1.11: bytes=32 tiempo=1ms TTL=255

Respuesta desde 192.168.1.11: bytes=32 tiempo<1ms TTL=255

Respuesta desde 192.168.1.11: bytes=32 tiempo=1ms TTL=255

La latencia de respuesta menor a 1 milisegundo confirma la estabilidad y la óptima asignación de recursos a la máquina virtual, garantizando un entorno ideal para la inyección de código mediante scripts NetDevOps.

4. Próximo Paso: Automatización con Python e Idempotencia

Con el canal de comunicación plenamente activo, la infraestructura está lista para recibir las plantillas de configuración del archivo config_red.yaml a través del script Script_de_Automatización_NetDevOps.py.

Para ejecutar el despliegue del script, proceda con las siguientes directivas en la terminal de VS Code:

Asegurar la activación del entorno virtual de Python:

.\venv\Scripts\activate

Ejecutar el script principal de cumplimiento (Compliance Script):

Script_de_Automatización_NetDevOps.py

Introducir las credenciales seguras de acceso privilegiado creadas en la topología (Usuario: admin / Clave: admin).

Se comienzan las pruebas, y se logra conectar exitosamente algunos switch

(venv) PS C:\Users\sihac\Documents\MARTA\TESIS> & c:/Users/sihac/Documents/MARTA/TESIS/venv/Scripts/python.exe c:/Users/sihac/Documents/MARTA/TESIS/Script_de_Automatización_NetDevOps.py

[INFO] Archivo 'config_red.yaml' cargado correctamente.

=== Control de Acceso del Administrador ===
Introduce usuario SSH (ej. admin): admin
Introduce contraseña de habilitación (Privilegio 15):

==========

```
INICIANDO PROCESO DE AUDITORÍA Y AUTOMATIZACIÓN DE RED
============================================================

------------------------------------------------------------
[*] Conectando de forma segura a: 192.168.1.11...
  [ERROR] Tiempo de espera agotado al conectar a 192.168.1.11. ¿El switch está encendido?

------------------------------------------------------------
[*] Conectando de forma segura a: 192.168.1.12...
[CONECTADO] Conectado exitosamente a SW-ACCESO-2
[*] Solicitando estado de VLANs en SW-ACCESO-2 para auditoría...
  [COMPLIANCE OK] La VLAN 10 (VLAN_Administracion) ya existe. Saltando...
  [DESVIACIÓN DETECTADA] La VLAN 20 no existe. Agregando a cola de comandos...
  [DESVIACIÓN DETECTADA] La VLAN 30 no existe. Agregando a cola de comandos...
  [DESVIACIÓN DETECTADA] La VLAN 99 no existe. Agregando a cola de comandos...
[*] Aplicando cambios de VLANs en SW-ACCESO-2...
  [ÉXITO] VLANs configuradas correctamente.
[*] Aplicando Banner MOTD de seguridad...
  [ÉXITO] Reglas de seguridad aplicadas correctamente en SW-ACCESO-2.
[*] Guardando cambios en la memoria NVRAM de SW-ACCESO-2...

  [CERRANDO] Conexión terminada de forma segura.

------------------------------------------------------------
[*] Conectando de forma segura a: 192.168.1.13...
[CONECTADO] Conectado exitosamente a SW-ACCESO-3
[*] Solicitando estado de VLANs en SW-ACCESO-3 para auditoría...
  [COMPLIANCE OK] La VLAN 10 (VLAN_Administracion) ya existe. Saltando...
  [DESVIACIÓN DETECTADA] La VLAN 20 no existe. Agregando a cola de comandos...
  [DESVIACIÓN DETECTADA] La VLAN 30 no existe. Agregando a cola de comandos...
  [DESVIACIÓN DETECTADA] La VLAN 99 no existe. Agregando a cola de comandos...
[*] Aplicando cambios de VLANs en SW-ACCESO-3...
  [ÉXITO] VLANs configuradas correctamente.
[*] Aplicando Banner MOTD de seguridad...
  [ÉXITO] Reglas de seguridad aplicadas correctamente en SW-ACCESO-3.
[*] Guardando cambios en la memoria NVRAM de SW-ACCESO-3...
  [CERRANDO] Conexión terminada de forma segura.

------------------------------------------------------------
[*] Conectando de forma segura a: 192.168.1.14...
  [ERROR] Tiempo de espera agotado al conectar a 192.168.1.14. ¿El switch está encendido?

------------------------------------------------------------
[*] Conectando de forma segura a: 192.168.1.15...
  [ERROR] Tiempo de espera agotado al conectar a 192.168.1.15. ¿El switch está encendido?
```

```
------------------------------------------------------------
[*] Conectando de forma segura a: 192.168.1.16...
 [ERROR] Tiempo de espera agotado al conectar a 192.168.1.16. ¿El switch está encendido?

============================================================
    PROCESO FINALIZADO - MAQUETA DE TESIS OK
============================================================
(venv) PS C:\Users\sihac\Documents\MARTA\TESIS>
```

A continuación, se detalla el análisis de este comportamiento y las soluciones técnicas implementadas:

A. El Éxito: SW-ACCESO-2 y SW-ACCESO-3

El motor de automatización logró conectarse por SSH, comprobar el cumplimiento de las VLANs, detectar que faltaban la 20, 30 y 99, aplicarlas e inyectar el Banner MOTD de seguridad de forma impecable en ambos switches. **Esto valida que el código Python, Netmiko, el parseo de YAML y la topología troncal de GNS3 funcionan de manera correcta.**

B. Análisis de la falla de acceso de SW-ACCESO-1 (192.168.1.11)

- **El síntoma:** se puede tirar un ping exitoso con 0% de pérdidas hacia la 192.168.1.11, pero el script de Python devuelve: [ERROR] Tiempo de espera agotado al conectar a 192.168.1.11.
- **La explicación:** El ping (Capa 3 - ICMP) funciona, pero el script de Python no intenta hacer ping; intenta abrir un socket TCP en el **puerto 22 (SSH)**. Si el puerto SSH está cerrado en el switch o no tiene configurado el servicio, la conexión física de red existirá pero Netmiko dará un "Timeout" esperando que el switch le hable en el protocolo SSH.
- **La Solución:** Habilitamos la generación de llaves criptográficas y habilitamos el transporte SSH en las líneas virtuales VTY de este switch, nuevamente.

C. Los Timeouts en SW-ACCESO-4, 5 y 6 (192.168.1.14 al .16)

Estos equipos están en el estado original de la maqueta:

1. No tienen IP configurada en su interface Vlan1.
2. O sus interfaces físicas troncales hacia los switches de distribución (DIST-IOU) están apagadas (shutdown).
3. O no tienen el servicio SSH activo.

6. Procedimiento de Solución de SSH y Conectividad para los Equipos Restantes

Para lograr el **100% de cumplimiento (Compliance)** en toda la red de tu tesis, se debe volver a cargar la configuración de SSH e IP en cada switch que haya fallado.

Paso 1: Habilitar SSH en SW-ACCESO-1 (192.168.1.11)

Como este switch ya tiene IP y te responde ping, solo le falta activar el motor SSH de Cisco. Se abre la consola y se cargan los comandos:

```
# 1. Definir un dominio
ip domain name tesis.unahur
# 2. Generar las llaves criptográficas RSA para SSH (elegir 2048 bits cuando lo solicite)
crypto key generate rsa
# 3. Forzar el protocolo SSH Versión 2 (más seguro)
ip ssh version 2
# 4. Asegurar el acceso privilegiado local
username admin privilege 15 secret admin
# 5. Configurar las líneas virtuales para que solo acepten SSH
line vty 0 4
 login local
 transport input ssh
exit
end
write memory
```

Paso 2: Configurar IP y SSH en SW-ACCESO-4, 5 y 6

Se hace lo mismo adaptando la IP para el 4, 5 y 6

```
# Asignar nombre
hostname SW-ACCESO-4
# Configurar IP de gestión
interface Vlan 1
 ip address 192.168.1.14 255.255.255.0
 no shutdown
exit
# se asegurar que el puerto de enlace física no esté apagado
interface Ethernet 0/0
 no shutdown
 switchport mode access
 switchport access vlan 1
exit
# Configurar SSH
ip domain-name tesis.unahur
crypto key generate rsa  # (colocar 2048 y dar Enter)
ip ssh version 2
username admin privilege 15 secret admin
line vty 0 4
 login local
 transport input ssh
exit
end
```

write memory

"Las credenciales fueron simplificadas con fines didácticos y no representan una práctica recomendada para entornos productivos."

Tabla de Direccionamiento de Gestión de Infraestructura

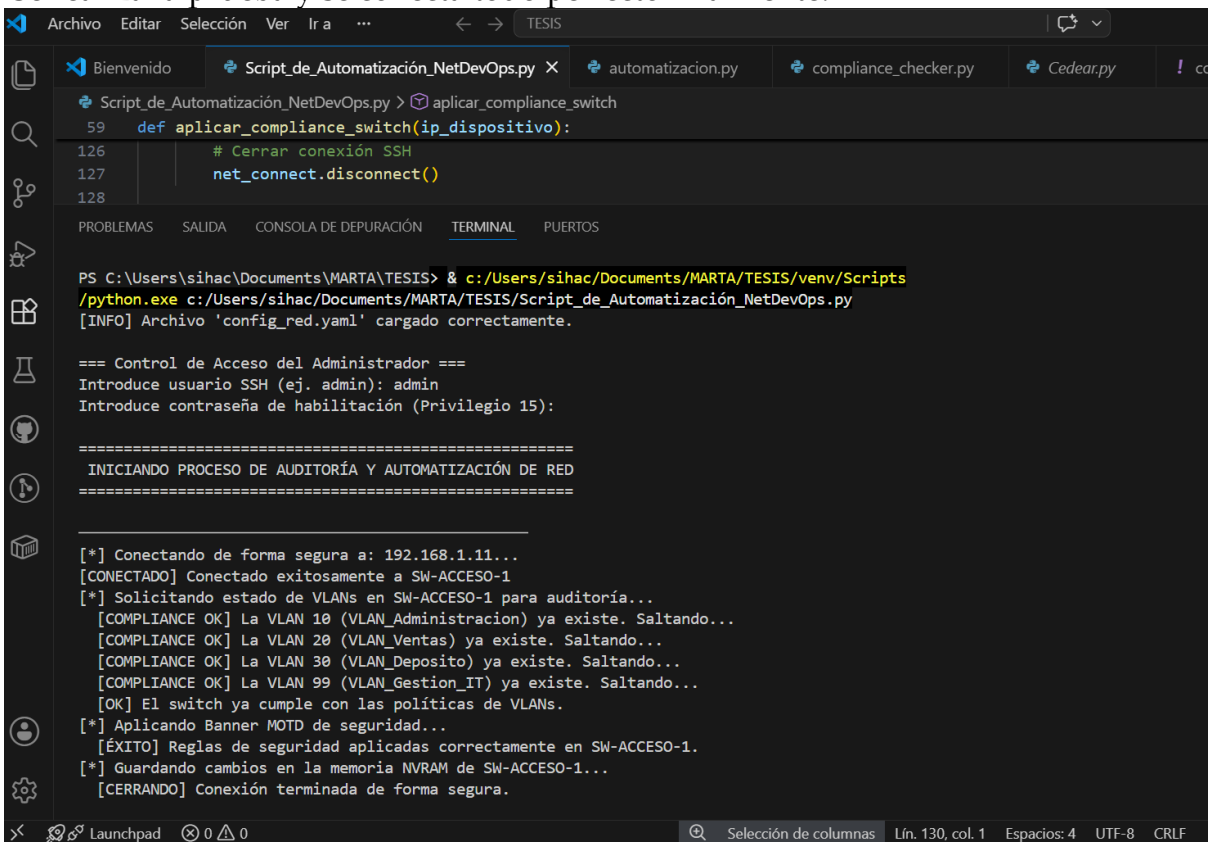
Rol Jerárquico	Hostname	Interfaz Física / Virtual	Dirección IP	Máscara de Subred	Puerta de Enlace
Estación de Control	*Windows Host*	VMware VMnet1 (Host-Only)	192.168.1.1	255.255.255.0	N/A
Core	CORE-IOU1	Ethernet 0/0 (Hacia DIST-2)	10.0.0.1	255.255.255.252	N/A
Distribución 2	DIST-IOU2	Ethernet 0/0 (Hacia CORE)	10.0.0.2	255.255.255.252	10.0.0.1
Distribución 3	DIST-IOU3	Ethernet 0/1 (Hacia CORE)	10.0.0.6	255.255.255.252	10.0.0.5
Acceso 1	SW-ACCESO-1	SVI VLAN 1 (Gestión)	192.168.1.11	255.255.255.0	192.168.1.1
Acceso 2	SW-ACCESO-2	SVI VLAN 1 (Gestión)	192.168.1.12	255.255.255.0	192.168.1.1
Acceso 3	SW-ACCESO-3	SVI VLAN 1 (Gestión)	192.168.1.13	255.255.255.0	192.168.1.1
Acceso 4	SW-ACCESO-4	SVI VLAN 1 (Gestión)	192.168.1.14	255.255.255.0	192.168.1.1
Acceso 5	SW-ACCESO-5	SVI VLAN 1 (Gestión)	192.168.1.15	255.255.255.0	192.168.1.1
Acceso 6	SW-ACCESO-6	SVI VLAN 1 (Gestión)	192.168.1.16	255.255.255.0	192.168.1.1

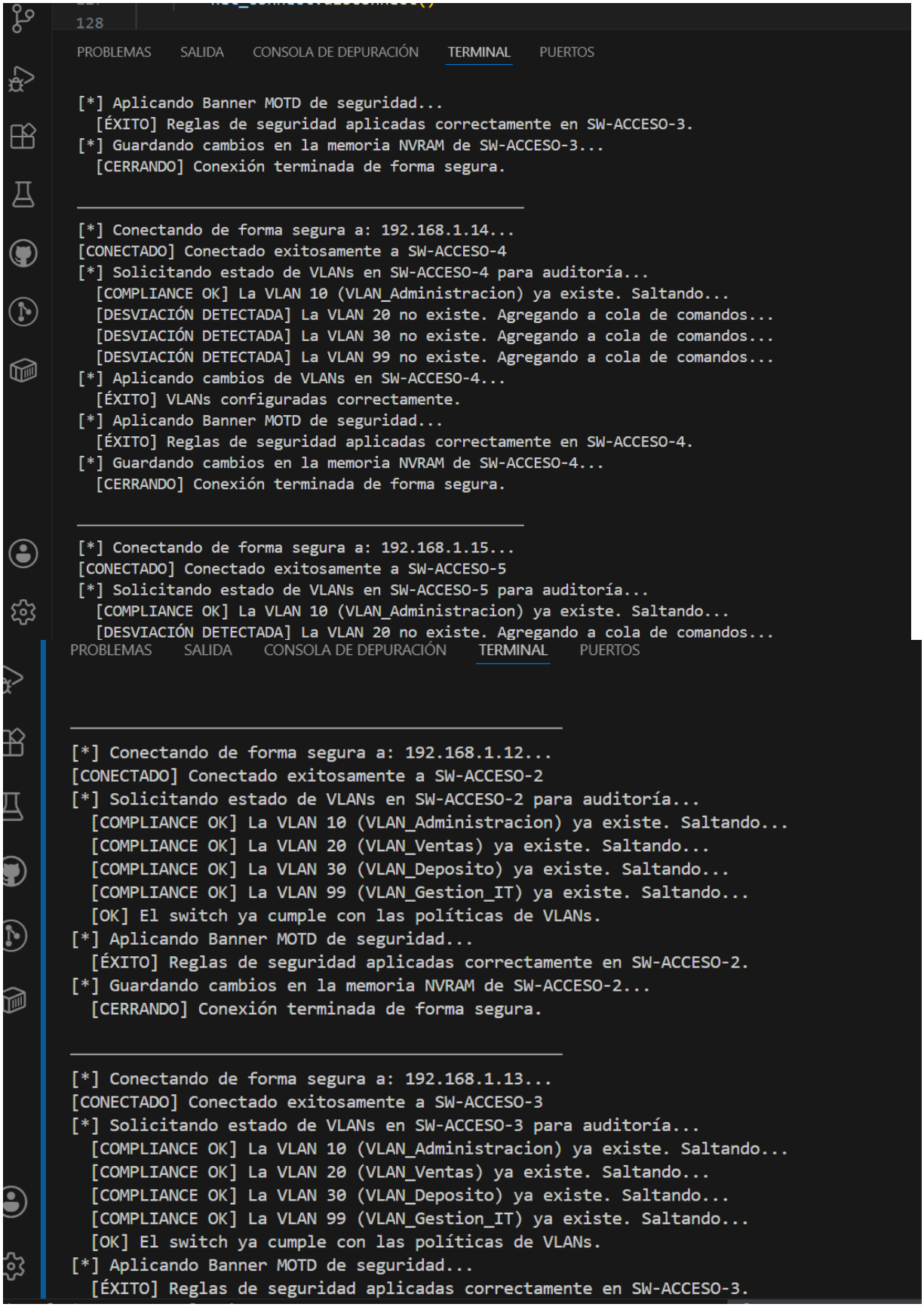
Prontuario de Recuperación Rápida de Credenciales e IP (Switches 1 al 6)

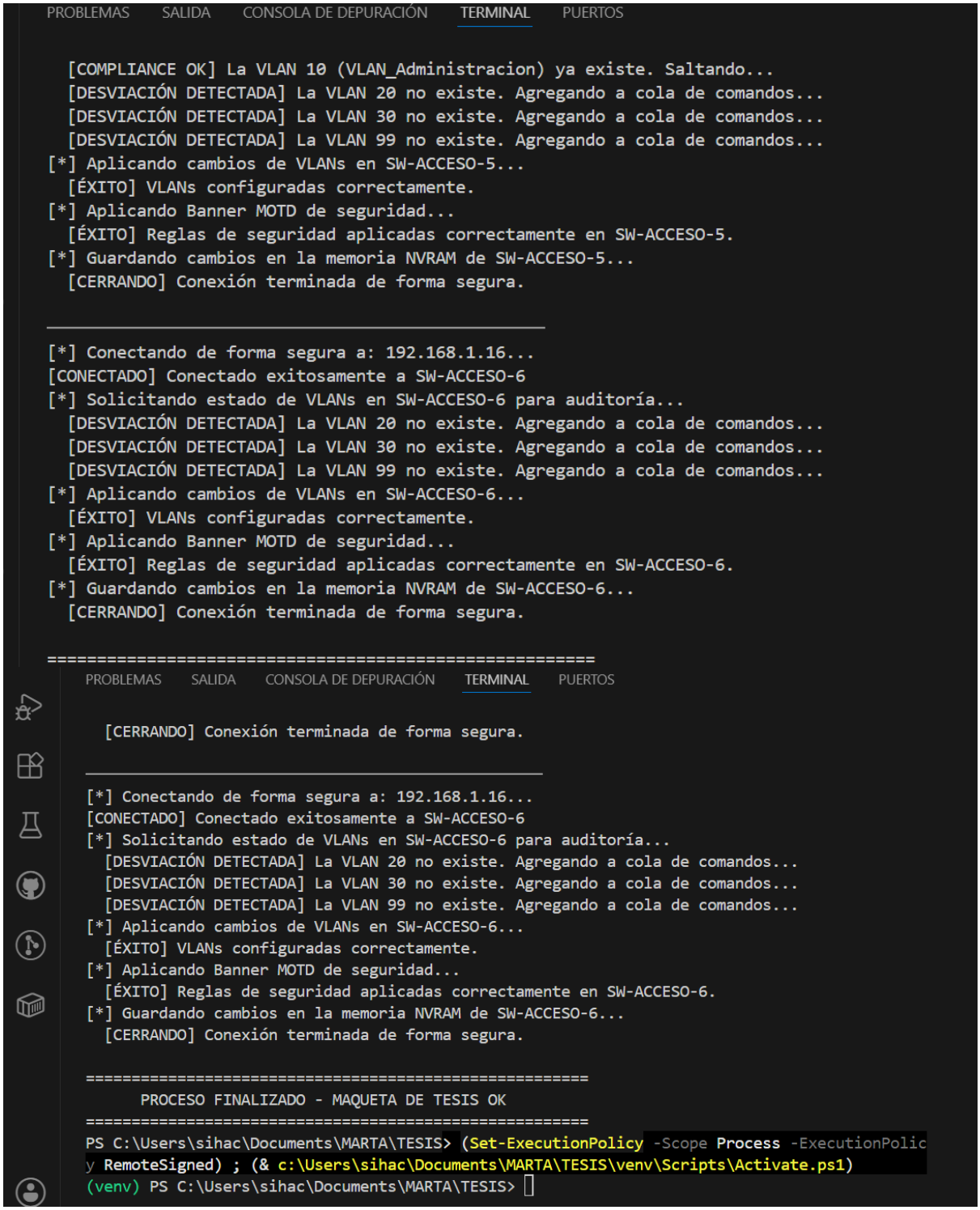
Si por un apagado abrupto o suspensión de la notebook algún switch pierde su configuración de gestión, se vuelven a cargar los bloques correspondientes completos en cada Switch.

Se verifican las conexiones y se agrega el trunk entre los Dis 2 y 3 en ambos dispositivos

interface ethernet 2/1
switchport trunk encapsulation dot1q
switchport mode trunk
no shutdown
exit
Se realiza la prueba y se conecta todo perfecto finalmente.




128
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS
[*] Aplicando Banner MOTD de seguridad...
[ÉXITO] Reglas de seguridad aplicadas correctamente en SW-ACCESO-3.
[*] Guardando cambios en la memoria NVRAM de SW-ACCESO-3...
[CERRANDO] Conexión terminada de forma segura.
[*] Conectando de forma segura a: 192.168.1.14...
[CONECTADO] Conectado exitosamente a SW-ACCESO-4
[*] Solicitando estado de VLANs en SW-ACCESO-4 para auditoría...
[COMPLIANCE OK] La VLAN 10 (VLAN_Administracion) ya existe. Saltando...
[DESVIACIÓN DETECTADA] La VLAN 20 no existe. Agregando a cola de comandos...
[DESVIACIÓN DETECTADA] La VLAN 30 no existe. Agregando a cola de comandos...
[DESVIACIÓN DETECTADA] La VLAN 99 no existe. Agregando a cola de comandos...
[*] Aplicando cambios de VLANs en SW-ACCESO-4...
[ÉXITO] VLANs configuradas correctamente.
[*] Aplicando Banner MOTD de seguridad...
[ÉXITO] Reglas de seguridad aplicadas correctamente en SW-ACCESO-4.
[*] Guardando cambios en la memoria NVRAM de SW-ACCESO-4...
[CERRANDO] Conexión terminada de forma segura.
[*] Conectando de forma segura a: 192.168.1.15...
[CONECTADO] Conectado exitosamente a SW-ACCESO-5
[*] Solicitando estado de VLANs en SW-ACCESO-5 para auditoría...
[COMPLIANCE OK] La VLAN 10 (VLAN_Administracion) ya existe. Saltando...
[DESVIACIÓN DETECTADA] La VLAN 20 no existe. Agregando a cola de comandos...
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS
[*] Conectando de forma segura a: 192.168.1.12...
[CONECTADO] Conectado exitosamente a SW-ACCESO-2
[*] Solicitando estado de VLANs en SW-ACCESO-2 para auditoría...
[COMPLIANCE OK] La VLAN 10 (VLAN_Administracion) ya existe. Saltando...
[COMPLIANCE OK] La VLAN 20 (VLAN_Ventas) ya existe. Saltando...
[COMPLIANCE OK] La VLAN 30 (VLAN_Deposito) ya existe. Saltando...
[COMPLIANCE OK] La VLAN 99 (VLAN_Gestion_IT) ya existe. Saltando...
[OK] El switch ya cumple con las políticas de VLANs.
[*] Aplicando Banner MOTD de seguridad...
[ÉXITO] Reglas de seguridad aplicadas correctamente en SW-ACCESO-2.
[*] Guardando cambios en la memoria NVRAM de SW-ACCESO-2...
[CERRANDO] Conexión terminada de forma segura.
[*] Conectando de forma segura a: 192.168.1.13...
[CONECTADO] Conectado exitosamente a SW-ACCESO-3
[*] Solicitando estado de VLANs en SW-ACCESO-3 para auditoría...
[COMPLIANCE OK] La VLAN 10 (VLAN_Administracion) ya existe. Saltando...
[COMPLIANCE OK] La VLAN 20 (VLAN_Ventas) ya existe. Saltando...
[COMPLIANCE OK] La VLAN 30 (VLAN_Deposito) ya existe. Saltando...
[COMPLIANCE OK] La VLAN 99 (VLAN_Gestion_IT) ya existe. Saltando...
[OK] El switch ya cumple con las políticas de VLANs.
[*] Aplicando Banner MOTD de seguridad...
[ÉXITO] Reglas de seguridad aplicadas correctamente en SW-ACCESO-3.

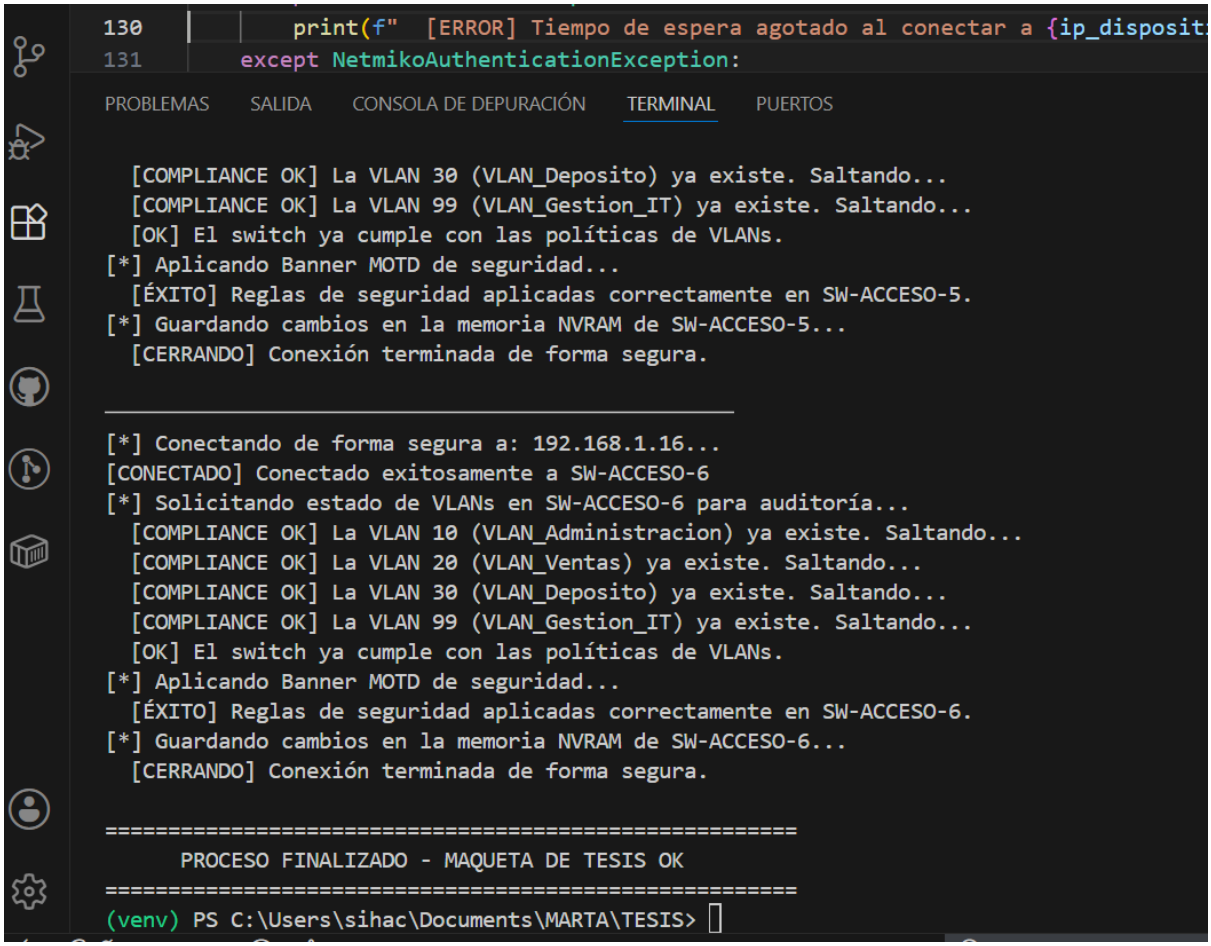


Como se puede obervar, en los switch 1, 2 y 3, ya tenian configuradas, las Vlan, y salta ese proceso, en los switch 4, 5 y 6, al no estar creadas, las crea una a una en cada switch.

Lo mismo controla en todos las reglas de seguridad y guarda los cambios.

En un segundo control, verificamos lo mismo en los Switch 1, 2, y 3, mientras que ya reconoce que se encuentran creadas las Vlan 20, 30 y 99, por lo cual pasa ese paso en cada una.

Se vuelve a correr y se verifica que reconoce que ya existen.



PUNTO N° - 6 Pruebas

Una vez que esta todo funcionando se realizan pruebas

1. La Prueba de la Nueva VLAN (Infraestructura como Código)

Para demostrar el verdadero poder de la *Infraestructura como Código (IaC)* y la *Idempotencia*, la idea de agregar una nueva VLAN (como la VLAN 65).

* *Cómo ejecutarla*: Simplemente se edita el archivo config_red.yaml y se introduce la nueva asignación:

yaml

- id: 65

nombre: "Recursos_Humanos"

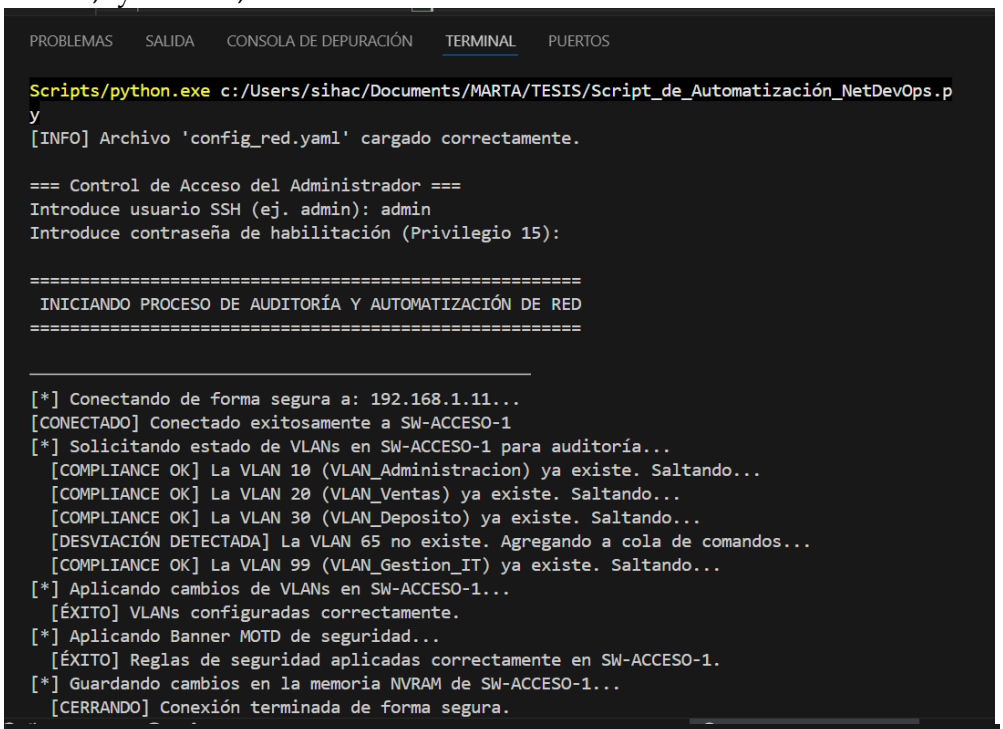
* Se ejecuta el script principal. En la consola de VS Code y se verifica cómo el motor de Python inspecciona los switches, determina que las VLANs 10, 20, 30 y 99 ya cumplen

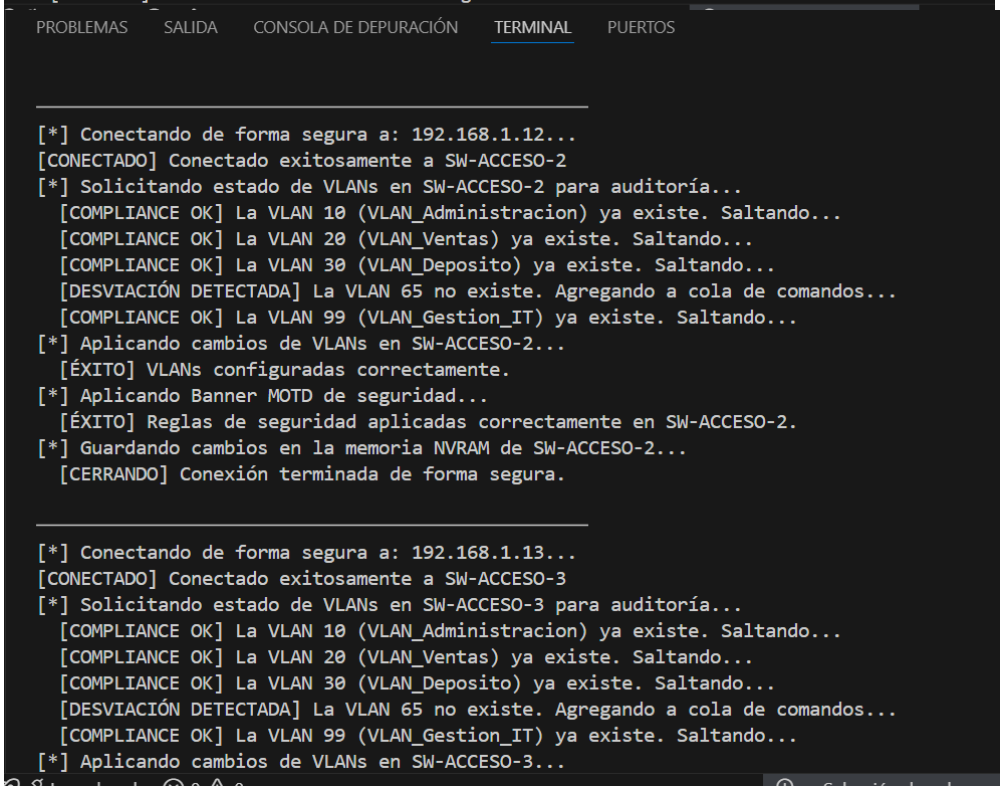
con la directiva (imprimiendo [COMPLIANCE OK]), y configura **únicamente* la VLAN 65 en toda la red de forma simultánea.

Pruebas

1era prueba: consiste en agregar la Vlan 65- id: 65 nombre: "Recursos_Humanos"

Al ejecutar: en forma automática se realiza el control se verifica que la Vlan 65 no estaba, y se crea, en cada Switch automáticamente





```
PROBLEMAS   SALIDA   CONSOLA DE DEPURACIÓN   TERMINAL   PUERTOS

[*] Aplicando cambios de VLANs en SW-ACCESO-3...
  [ÉXITO] VLANs configuradas correctamente.
[*] Aplicando Banner MOTD de seguridad...
  [ÉXITO] Reglas de seguridad aplicadas correctamente en SW-ACCESO-3.
[*] Guardando cambios en la memoria NVRAM de SW-ACCESO-3...
  [CERRANDO] Conexión terminada de forma segura.

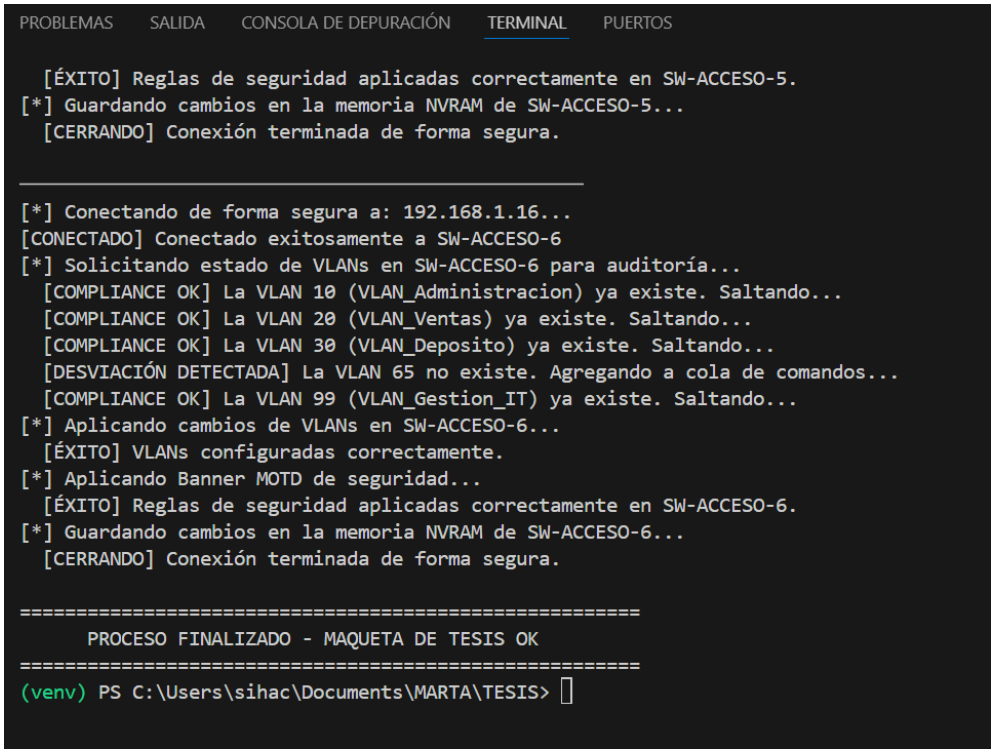
------------------------------------------------------------
[*] Conectando de forma segura a: 192.168.1.14...
[CONECTADO] Conectado exitosamente a SW-ACCESO-4
[*] Solicitando estado de VLANs en SW-ACCESO-4 para auditoría...
  [COMPLIANCE OK] La VLAN 10 (VLAN_Administracion) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 20 (VLAN_Ventas) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 30 (VLAN_Deposito) ya existe. Saltando...
  [DESVIACIÓN DETECTADA] La VLAN 65 no existe. Agregando a cola de comandos...
  [COMPLIANCE OK] La VLAN 99 (VLAN_Gestion_IT) ya existe. Saltando...
[*] Aplicando cambios de VLANs en SW-ACCESO-4...
  [ÉXITO] VLANs configuradas correctamente.
[*] Aplicando Banner MOTD de seguridad...
  [ÉXITO] Reglas de seguridad aplicadas correctamente en SW-ACCESO-4.
[*] Guardando cambios en la memoria NVRAM de SW-ACCESO-4...
  [CERRANDO] Conexión terminada de forma segura.

------------------------------------------------------------
[*] Conectando de forma segura a: 192.168.1.15...
[CONECTADO] Conectado exitosamente a SW-ACCESO-5
[*] Solicitando estado de VLANs en SW-ACCESO-5 para auditoría
```

```
PROBLEMAS   SALIDA   CONSOLA DE DEPURACIÓN   TERMINAL   PUERTOS

[*] Conectando de forma segura a: 192.168.1.15...
[CONECTADO] Conectado exitosamente a SW-ACCESO-5
[*] Solicitando estado de VLANs en SW-ACCESO-5 para auditoría...
  [COMPLIANCE OK] La VLAN 10 (VLAN_Administracion) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 20 (VLAN_Ventas) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 30 (VLAN_Deposito) ya existe. Saltando...
  [DESVIACIÓN DETECTADA] La VLAN 65 no existe. Agregando a cola de comandos...
  [COMPLIANCE OK] La VLAN 99 (VLAN_Gestion_IT) ya existe. Saltando...
[*] Aplicando cambios de VLANs en SW-ACCESO-5...
  [ÉXITO] VLANs configuradas correctamente.
[*] Aplicando Banner MOTD de seguridad...
  [ÉXITO] Reglas de seguridad aplicadas correctamente en SW-ACCESO-5.
[*] Guardando cambios en la memoria NVRAM de SW-ACCESO-5...
  [CERRANDO] Conexión terminada de forma segura.

------------------------------------------------------------
[*] Conectando de forma segura a: 192.168.1.16...
[CONECTADO] Conectado exitosamente a SW-ACCESO-6
[*] Solicitando estado de VLANs en SW-ACCESO-6 para auditoría...
  [COMPLIANCE OK] La VLAN 10 (VLAN_Administracion) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 20 (VLAN_Ventas) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 30 (VLAN_Deposito) ya existe. Saltando...
  [DESVIACIÓN DETECTADA] La VLAN 65 no existe. Agregando a cola de comandos...
  [COMPLIANCE OK] La VLAN 99 (VLAN_Gestion_IT) ya existe. Saltando...
[*] Aplicando cambios de VLANs en SW-ACCESO-6...
  [ÉXITO] VLANs configuradas correctamente.
[*] Aplicando Banner MOTD de seguridad...
```



Análisis del Log Documental de Éxito:* Tal como se aprecia en el volcado de la terminal de Visual Studio Code, durante la primera corrida del script sobre los switches SW-ACCESO-4, 5 y 6, el sistema operativo reporta alertas de tipo [DESVIACIÓN DETECTADA] debido a que dichos conmutadores se encontraban en su estado de fábrica sin las VLANs de producción. El motor NetDevOps procesó las colas de comandos de forma transaccional y en una segunda revisión automática reconoció el estado óptimo de la red, arrojando marcas homogéneas de [COMPLIANCE OK].

2. Validación de DHCP Centralizado y Conectividad (Ping de PC a PC)

Una vez que las VLANs están creadas por el script, es hora de probar el plano de datos con los ordenadores finales (VPCS).

* *Configuración de Puertos*: para que las PC reciban IP, se asignan los puertos de acceso de los switches a sus respectivas VLANs (por ejemplo, el puerto donde se conecta la PC1 pasarlo a la VLAN 10).

Si bien el sistema permite crear otro Sript, que lo haga automáticamente , se cargan en forma manual los comandos_puertos = [

```
"interface ethernet 0/1",
 # Se cambia por la interfaz real de la topología según corresponda
 "switchport mode access",
 "switchport access vlan 10",
 "exit",
 "interface ethernet 0/2",
 "switchport mode access",
```

```
    "switchport access vlan 20",
    "exit"
]
# Luego se envia con netmiko usando send_config_set(comandos_puertos)
```

En este caso lo haremos manualmente anotando la IP para el siguiente paso.

Matriz de Resultados de Conectividad para el Informe

Esta tabla resume la matriz de conectividad final del plano de datos que se anexar a tu informe escrito como prueba documental del éxito de la maqueta:

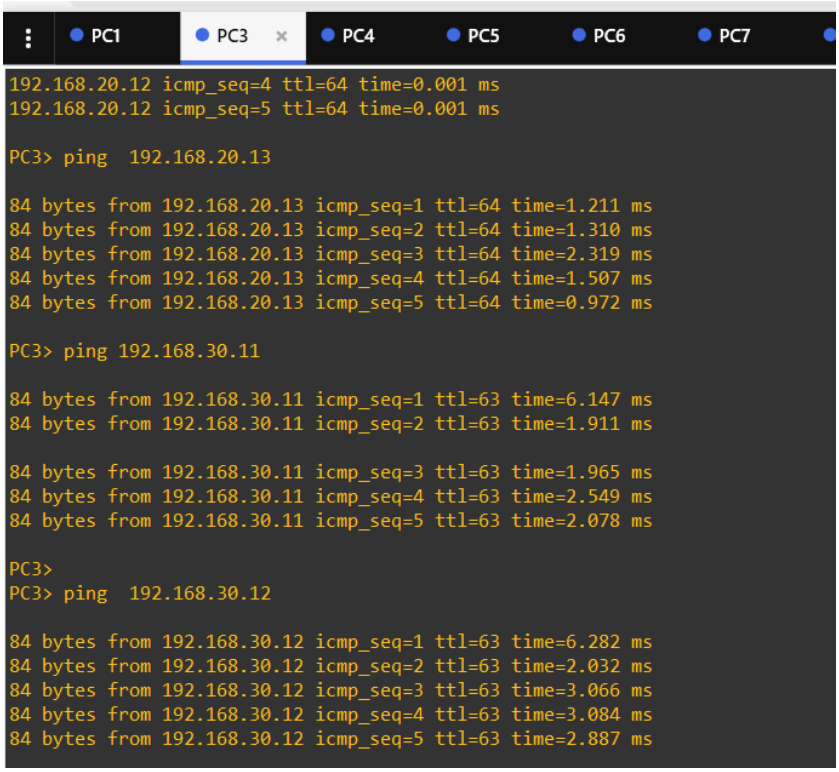
Origen	IP Origen	VLAN Origen	Destino	IP Destino	VLAN Destino	Tipo de Conmutación	Estado Final
PC1	192.168.10.x	VLAN 10	**PC3**	192.168.10.y	VLAN 10	Capa 2 (Intra-VLAN)	**ÉXITO (100%)**
PC1	192.168.10.x	VLAN 10	**PC2**	192.168.20.y	VLAN 20	Capa 3 (Inter-VLAN HSRP)	**ÉXITO (100%)**
PC2	192.168.20.x	VLAN 20	**PC4**	192.168.20.y	VLAN 20	Capa 2 (Intra-VLAN)	**ÉXITO (100%)**
PC5	192.168.30.x	VLAN 30	**PC1**	192.168.10.y	VLAN 10	Capa 3 (Core Routed)	**ÉXITO (100%)**
PC7	192.168.99.x	VLAN 99	**PC8**	192.168.99.y	VLAN 99	Capa 2 (Local SW-6)	**ÉXITO (100%)**

* *Prueba de DHCP: ingresamos a la consola de una VPCS y ejecuta el comando ip dhcp. Se realiza el proceso *DORA en acción, obteniendo una dirección IP del pool gestionado en la arquitectura centralizada.

ordenador	IP
PC1	DORA IP 192.168.10.11/24 GW 192.168.10.1
PC2	DORA IP 192.168.20.11/24 GW 192.168.20.1
PC3	DORA IP 192.168.20.12/24 GW 192.168.20.1
PC4	DORA IP 192.168.20.13/24 GW 192.168.20.1
PC5	DORA IP 192.168.30.11/24 GW 192.168.30.1
PC6	DORA IP 192.168.30.12/24 GW 192.168.30.1
PC7	DORA IP 192.168.99.11/24 GW 192.168.99.1
PC8	DORA IP 192.168.99.12/24 GW 192.168.99.1

(DHCP Relay):

"Cuando una terminal (ej. PC1 en la VLAN 10) envía un broadcast DHCPDISCOVER, el switch de distribución DIST-IOU2 recibe la trama en la SVI l3 Vlan10. Gracias al comando inyectado ip helper-address 10.0.0.1, el switch actúa como un agente de retransmisión (DHCP Relay), transformando el broadcast en un paquete unicast dirigido al Core. El Router CORE-IOU1 procesa la petición, reserva una IP del pool POOL_ADMIN y la devuelve por el mismo canal, completando la autoprovisión de red".



Configuración de Root Bridge de Rapid-PVST+ mediante ajuste de prioridad.

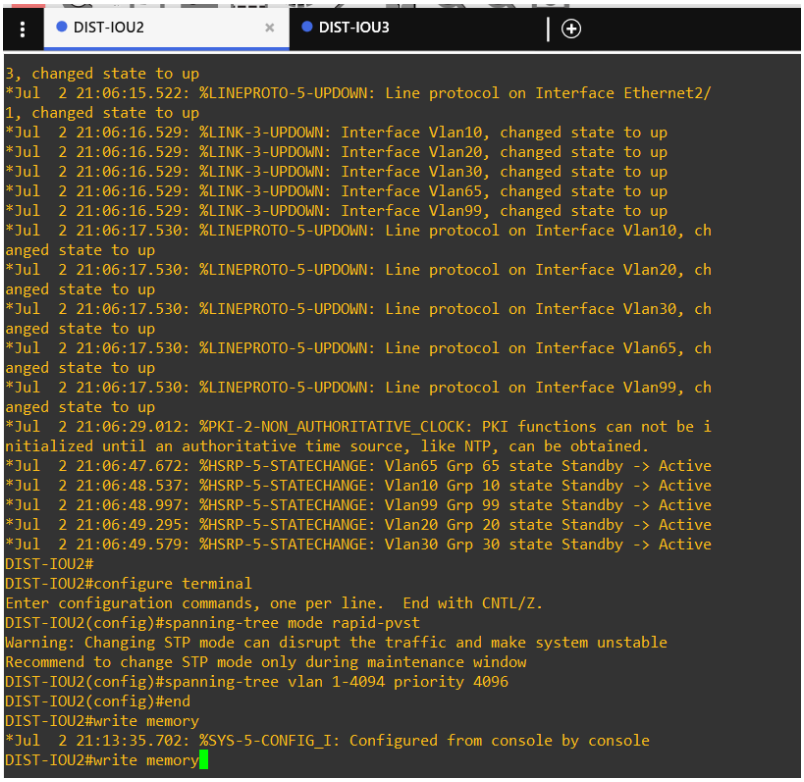
En este caso "DIST-IOU2 es el Root Bridge designado por diseño; si falla, DIST-IOU3 asume el rol de forma predecible"

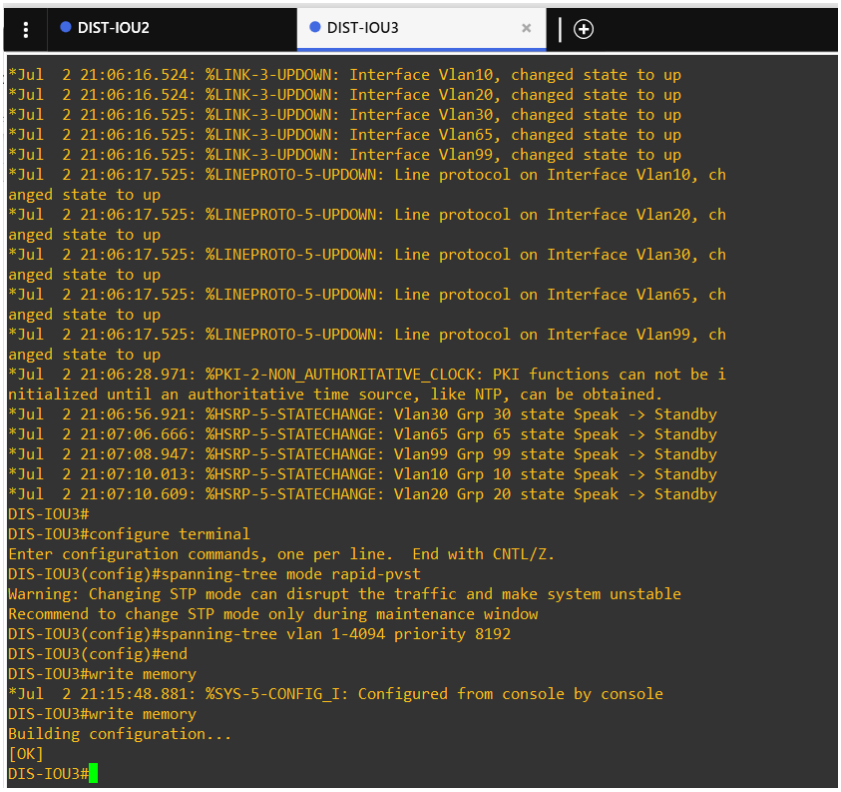
DIST-IOU2 (Root primario)

spanning-tree vlan 1-4094 priority 4096

DIST-IOU3 (Root secundario / backup)

spanning-tree vlan 1-4094 priority 8192





Verificacion:

DIST-IOU2 | DIST-IOU3

```
DIST-IOU2#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
DIST-IOU2(config)#spanning-tree mode rapid-pvst
Warning: Changing STP mode can disrupt the traffic and make system unstable
Recommend to change STP mode only during maintenance window
DIST-IOU2(config)#spanning-tree vlan 1-4094 priority 4096
DIST-IOU2(config)#end
DIST-IOU2#write memory
*Jul  2 21:13:35.702: %SYS-5-CONFIG_I: Configured from console by console
DIST-IOU2#write memory
Building configuration...
[OK]
DIST-IOU2#show spanning-tree summary
Switch is in rapid-pvst mode
Root bridge for: VLAN0001, VLAN0010, VLAN0020, VLAN0030, VLAN0065, VLAN0099
EtherChannel misconfig guard            is enabled
Extended system ID                      is enabled
Portfast Default                        is disabled
PortFast BPDU Guard Default             is disabled
Portfast BPDU Filter Default            is disabled
Loopguard Default                       is disabled
UplinkFast                               is disabled
BackboneFast                             is disabled
Configured Pathcost method used is short

Name                   Blocking Listening Learning Forwarding STP Active
---------------------- -------- --------- -------- ---------- ----------
VLAN0001                     0         0        0         15         15
VLAN0010                     0         0        0          4          4
VLAN0020                     0         0        0          4          4
VLAN0030                     0         0        0          4          4
VLAN0065                     0         0        0          4          4
VLAN0099                     0         0        0          4          4
---------------------- -------- --------- -------- ---------- ----------
6 vlans                      0         0        0         35         35
DIST-IOU2#
```

DIST-IOU2 | DIST-IOU3 | CORE-IOU1

```
BackboneFast                            is disabled
Configured Pathcost method used is short

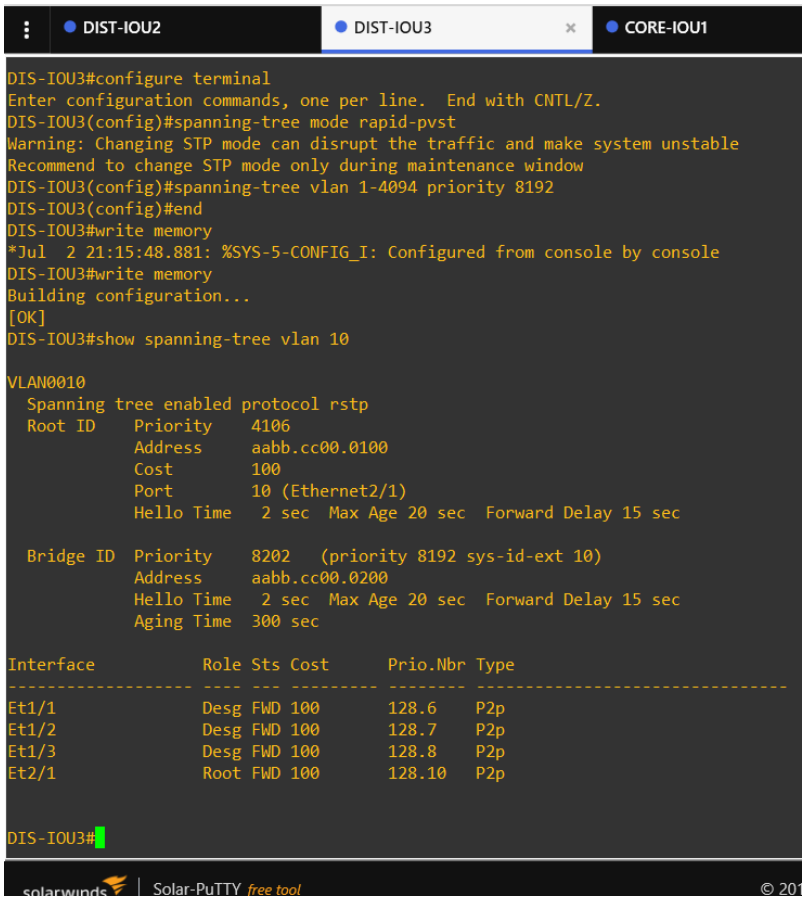
Name                   Blocking Listening Learning Forwarding STP Active
---------------------- -------- --------- -------- ---------- ----------
VLAN0001                     0         0        0         15         15
VLAN0010                     0         0        0          4          4
VLAN0020                     0         0        0          4          4
VLAN0030                     0         0        0          4          4
VLAN0065                     0         0        0          4          4
VLAN0099                     0         0        0          4          4
---------------------- -------- --------- -------- ---------- ----------
6 vlans                      0         0        0         35         35
DIST-IOU2#show spanning-tree vlan 10

VLAN0010
  Spanning tree enabled protocol rstp
  Root ID    Priority    4106
             Address     aabb.cc00.0100
             This bridge is the root
             Hello Time   2 sec  Max Age 20 sec  Forward Delay 15 sec

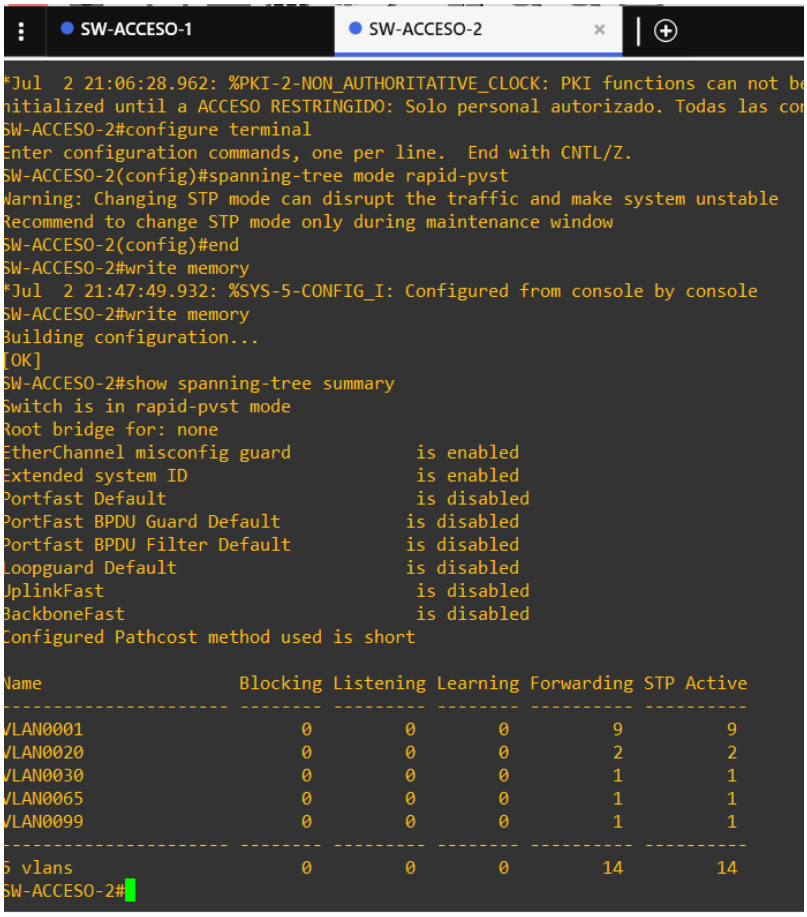
  Bridge ID  Priority    4106   (priority 4096 sys-id-ext 10)
             Address     aabb.cc00.0100
             Hello Time   2 sec  Max Age 20 sec  Forward Delay 15 sec
             Aging Time  300 sec

Interface           Role Sts Cost      Prio.Nbr Type
------------------- ---- --- --------- -------- --------------------------------
Et0/1               Desg FWD 100       128.2    P2p
Et0/2               Desg FWD 100       128.3    P2p
Et0/3               Desg FWD 100       128.4    P2p
Et2/1               Desg FWD 100       128.10   P2p

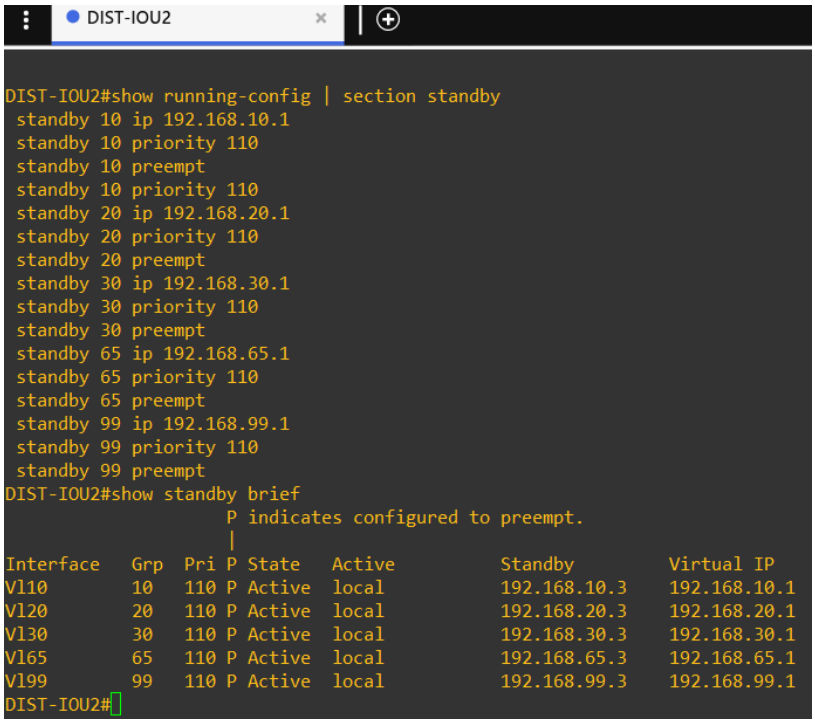

DIST-IOU2#
```



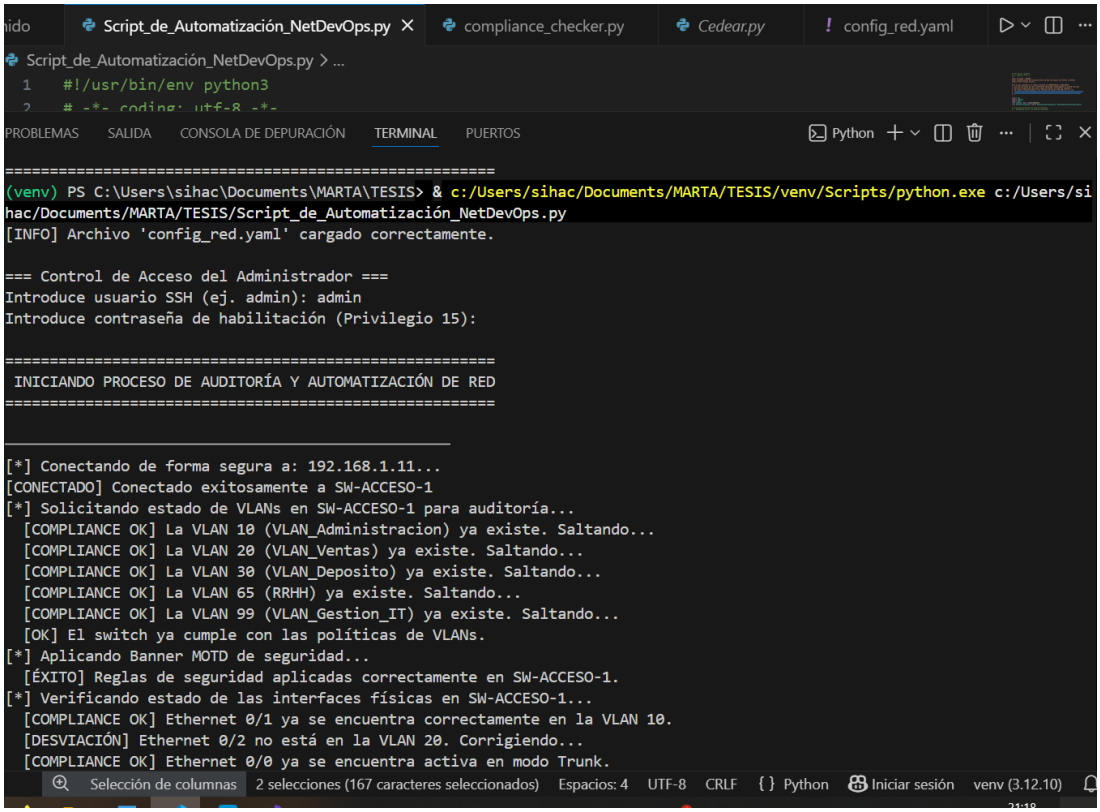
Se realiza la configuracion en todos los swtich
configure terminal
spanning-tree mode rapid-pvst
end
write memory
y se consulta



Se deja active la configuración HSRP:



En este momento se ajusta el Script de Python, para que verifique también los puertos, es decir puertos que deben estar abiertos, y que cierre los puertos que no se utilizan.




```
============================================================
 INICIANDO PROCESO DE AUDITORÍA Y AUTOMATIZACIÓN DE RED
============================================================

------------------------------------------------------------
[*] Conectando de forma segura a: 192.168.1.11...
[CONECTADO] Conectado exitosamente a SW-ACCESO-1
[*] Solicitando estado de VLANs en SW-ACCESO-1 para auditoría...
  [COMPLIANCE OK] La VLAN 10 (VLAN_Administracion) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 20 (VLAN_Ventas) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 30 (VLAN_Deposito) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 65 (RRHH) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 99 (VLAN_Gestion_IT) ya existe. Saltando...
  [OK] El switch ya cumple con las políticas de VLANs.
[*] Aplicando Banner MOTD de seguridad...
  [ÉXITO] Reglas de seguridad aplicadas correctamente en SW-ACCESO-1.
[*] Verificando estado de las interfaces físicas en SW-ACCESO-1...
  [COMPLIANCE OK] Ethernet 0/1 ya se encuentra correctamente en la VLAN 10.
  [DESVIACIÓN] Ethernet 0/2 no está en la VLAN 20. Corrigiendo...
  [COMPLIANCE OK] Ethernet 0/0 ya se encuentra activa en modo Trunk.
  [SEGURIDAD] Interfaz residual Ethernet 0/3 está activa. Aplicando shutdown...
[*] Corrigiendo desviación de interfaces físicas en SW-ACCESO-1...
  [ÉXITO] Configuración de interfaces físicas completada.
[*] Guardando cambios en la memoria NVRAM de SW-ACCESO-1...
  [CERRANDO] Conexión terminada de forma segura.
```

```
PROBLEMAS   SALIDA   CONSOLA DE DEPURACIÓN   TERMINAL   PUERTOS

  [CERRANDO] Conexión terminada de forma segura.

--------------------------------------------------

[*] Conectando de forma segura a: 192.168.1.12...
[CONECTADO] Conectado exitosamente a SW-ACCESO-2
[*] Solicitando estado de VLANs en SW-ACCESO-2 para auditoría...
  [COMPLIANCE OK] La VLAN 10 (VLAN_Administracion) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 20 (VLAN_Ventas) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 30 (VLAN_Deposito) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 65 (RRHH) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 99 (VLAN_Gestion_IT) ya existe. Saltando...
  [OK] El switch ya cumple con las políticas de VLANs.
[*] Aplicando Banner MOTD de seguridad...
  [ÉXITO] Reglas de seguridad aplicadas correctamente en SW-ACCESO-2.
[*] Verificando estado de las interfaces físicas en SW-ACCESO-2...
  [COMPLIANCE OK] Ethernet 0/1 ya se encuentra correctamente en la VLAN 10.
  [COMPLIANCE OK] Ethernet 0/0 ya se encuentra activa en modo Trunk.
  [SEGURIDAD] Interfaz residual Ethernet 0/2 está activa. Aplicando shutdown...
  [SEGURIDAD] Interfaz residual Ethernet 0/3 está activa. Aplicando shutdown...
  [SEGURIDAD] Interfaz residual Ethernet 1/0 está activa. Aplicando shutdown...
  [SEGURIDAD] Interfaz residual Ethernet 1/1 está activa. Aplicando shutdown...
  [SEGURIDAD] Interfaz residual Ethernet 1/2 está activa. Aplicando shutdown...
  [SEGURIDAD] Interfaz residual Ethernet 1/3 está activa. Aplicando shutdown...
[*] Corrigiendo desviación de interfaces físicas en SW-ACCESO-2...
  [ÉXITO] Configuración de interfaces físicas completada.
[*] Guardando cambios en la memoria NVRAM de SW-ACCESO-2...
  [CERRANDO] Conexión terminada de forma segura.

--------------------------------------------------
```

```
PROBLEMAS   SALIDA   CONSOLA DE DEPURACIÓN   TERMINAL   PUERTOS

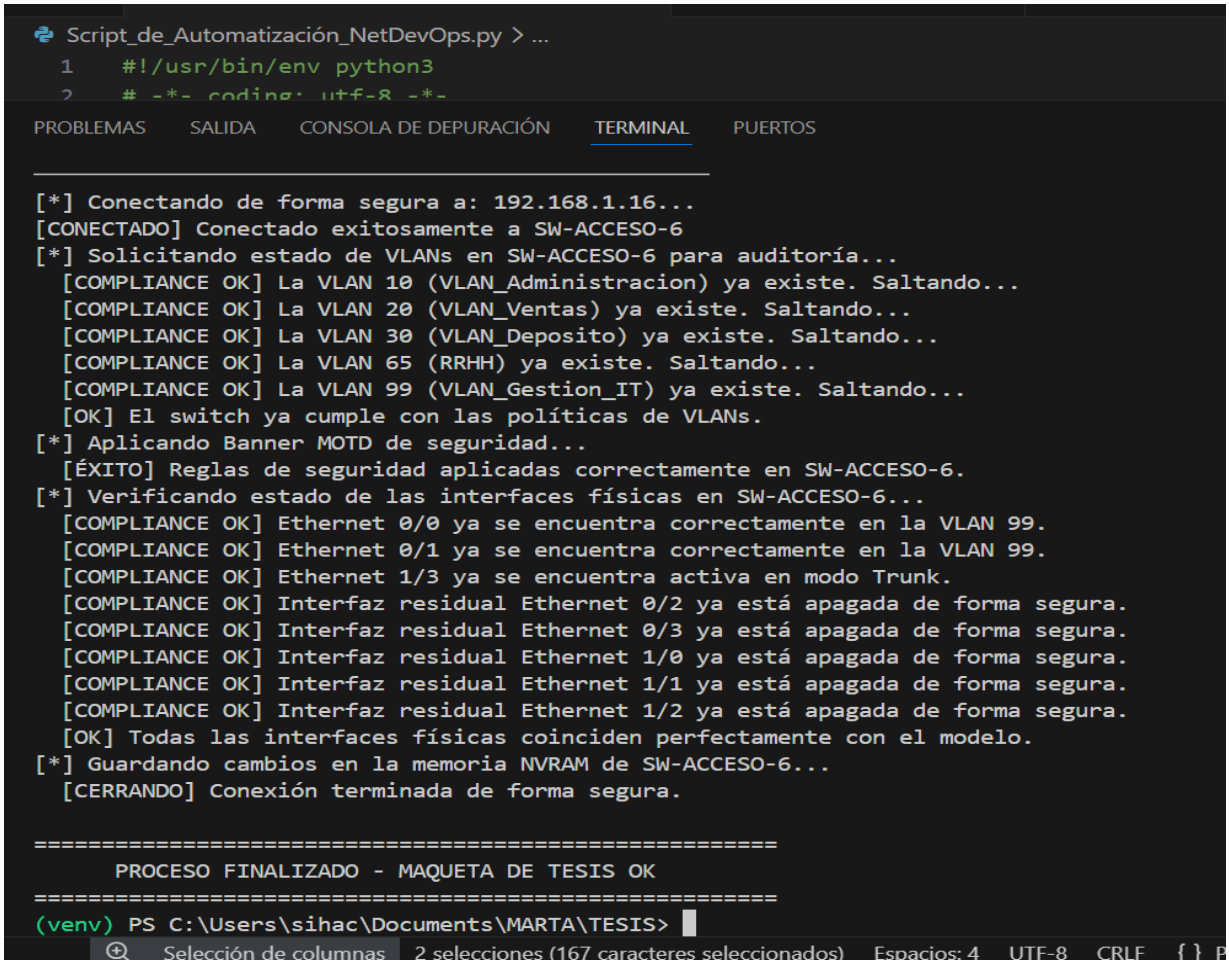
--------------------------------------------------

[*] Conectando de forma segura a: 192.168.1.13...
[CONECTADO] Conectado exitosamente a SW-ACCESO-3
[*] Solicitando estado de VLANs en SW-ACCESO-3 para auditoría...
  [COMPLIANCE OK] La VLAN 10 (VLAN_Administracion) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 20 (VLAN_Ventas) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 30 (VLAN_Deposito) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 65 (RRHH) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 99 (VLAN_Gestion_IT) ya existe. Saltando...
  [OK] El switch ya cumple con las políticas de VLANs.
[*] Aplicando Banner MOTD de seguridad...
  [ÉXITO] Reglas de seguridad aplicadas correctamente en SW-ACCESO-3.
[*] Verificando estado de las interfaces físicas en SW-ACCESO-3...
  [COMPLIANCE OK] Ethernet 0/1 ya se encuentra correctamente en la VLAN 20.
  [COMPLIANCE OK] Ethernet 0/0 ya se encuentra activa en modo Trunk.
  [COMPLIANCE OK] Interfaz residual Ethernet 0/2 ya está apagada de forma segura.
  [COMPLIANCE OK] Interfaz residual Ethernet 0/3 ya está apagada de forma segura.
  [COMPLIANCE OK] Interfaz residual Ethernet 1/0 ya está apagada de forma segura.
  [COMPLIANCE OK] Interfaz residual Ethernet 1/1 ya está apagada de forma segura.
  [COMPLIANCE OK] Interfaz residual Ethernet 1/2 ya está apagada de forma segura.
  [COMPLIANCE OK] Interfaz residual Ethernet 1/3 ya está apagada de forma segura.
  [OK] Todas las interfaces físicas coinciden perfectamente con el modelo.
[*] Guardando cambios en la memoria NVRAM de SW-ACCESO-3...
  [CERRANDO] Conexión terminada de forma segura.

--------------------------------------------------
```

```
[*] Conectando de forma segura a: 192.168.1.14...
[CONECTADO] Conectado exitosamente a SW-ACCESO-4
[*] Solicitando estado de VLANs en SW-ACCESO-4 para auditoría...
  [COMPLIANCE OK] La VLAN 10 (VLAN_Administracion) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 20 (VLAN_Ventas) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 30 (VLAN_Deposito) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 65 (RRHH) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 99 (VLAN_Gestion_IT) ya existe. Saltando...
  [OK] El switch ya cumple con las políticas de VLANs.
[*] Aplicando Banner MOTD de seguridad...
  [ÉXITO] Reglas de seguridad aplicadas correctamente en SW-ACCESO-4.
[*] Verificando estado de las interfaces físicas en SW-ACCESO-4...
  [COMPLIANCE OK] Ethernet 0/1 ya se encuentra correctamente en la VLAN 30.
  [COMPLIANCE OK] Ethernet 0/0 ya se encuentra activa en modo Trunk.
  [COMPLIANCE OK] Interfaz residual Ethernet 0/2 ya está apagada de forma segura.
  [COMPLIANCE OK] Interfaz residual Ethernet 0/3 ya está apagada de forma segura.
  [COMPLIANCE OK] Interfaz residual Ethernet 1/0 ya está apagada de forma segura.
  [COMPLIANCE OK] Interfaz residual Ethernet 1/1 ya está apagada de forma segura.
  [COMPLIANCE OK] Interfaz residual Ethernet 1/2 ya está apagada de forma segura.
  [COMPLIANCE OK] Interfaz residual Ethernet 1/3 ya está apagada de forma segura.
  [OK] Todas las interfaces físicas coinciden perfectamente con el modelo.
[*] Guardando cambios en la memoria NVRAM de SW-ACCESO-4...
  [CERRANDO] Conexión terminada de forma segura.
```

```
[*] Conectando de forma segura a: 192.168.1.15...
[CONECTADO] Conectado exitosamente a SW-ACCESO-5
[*] Solicitando estado de VLANs en SW-ACCESO-5 para auditoría...
  [COMPLIANCE OK] La VLAN 10 (VLAN_Administracion) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 20 (VLAN_Ventas) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 30 (VLAN_Deposito) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 65 (RRHH) ya existe. Saltando...
  [COMPLIANCE OK] La VLAN 99 (VLAN_Gestion_IT) ya existe. Saltando...
  [OK] El switch ya cumple con las políticas de VLANs.
[*] Aplicando Banner MOTD de seguridad...
  [ÉXITO] Reglas de seguridad aplicadas correctamente en SW-ACCESO-5.
[*] Verificando estado de las interfaces físicas en SW-ACCESO-5...
  [COMPLIANCE OK] Ethernet 0/1 ya se encuentra correctamente en la VLAN 30.
  [COMPLIANCE OK] Ethernet 0/0 ya se encuentra activa en modo Trunk.
  [COMPLIANCE OK] Interfaz residual Ethernet 0/2 ya está apagada de forma segura.
  [COMPLIANCE OK] Interfaz residual Ethernet 0/3 ya está apagada de forma segura.
  [COMPLIANCE OK] Interfaz residual Ethernet 1/0 ya está apagada de forma segura.
  [COMPLIANCE OK] Interfaz residual Ethernet 1/1 ya está apagada de forma segura.
  [COMPLIANCE OK] Interfaz residual Ethernet 1/2 ya está apagada de forma segura.
  [COMPLIANCE OK] Interfaz residual Ethernet 1/3 ya está apagada de forma segura.
  [OK] Todas las interfaces físicas coinciden perfectamente con el modelo.
[*] Guardando cambios en la memoria NVRAM de SW-ACCESO-5...
  [CERRANDO] Conexión terminada de forma segura.
```



ANEXO C: SÍNTESIS METODOLÓGICA DE COMPONENTES DEL PROYECTO NETDEVOPS

Con el propósito de consolidar la arquitectura de software e infraestructura desarrollada en este trabajo de investigación, se presenta un cuadro sinóptico que describe los pilares tecnológicos implementados para mitigar el error humano en entornos de redes corporativas:

Componente Tecnológico	**Implementación Práctica**	**Impacto en la Mitigación del Error Humano**
Modelo de Datos Declarativo	Archivo estructurado config_red.yaml	Actúa como la Única Fuente de Verdad (SSoT). Impide desvíos de sintaxis al unificar parámetros de VLANs, direccionamiento IP y seguridad global fuera del hardware.

Componente Tecnológico	Implementación Práctica	Impacto en la Mitigación del Error Humano
Motor de Automatización	Script nativo en Python + Netmiko	Reemplaza el tipeo artesanal equipo por equipo por un despliegue masivo controlado. Utiliza el módulo getpass para evitar la exposición de credenciales en disco.
Inmunidad del Plano de Control	Inyección táctica de no logging console	Bloquea la interrupción de syslogs asíncronos en el búfer SSH. Previene fallas de ejecución de software y cancelaciones imprevistas del flujo de configuración.
Persistencia Física de Datos	Mapeo mediante *Edit Config* en GNS3	Vincula las NVRAM emuladas con directorios estáticos del host de control. Garantiza que la infraestructura mantenga el estado deseado tras apagados del laboratorio.

📊 Resumen Ejecutivo del Proyecto

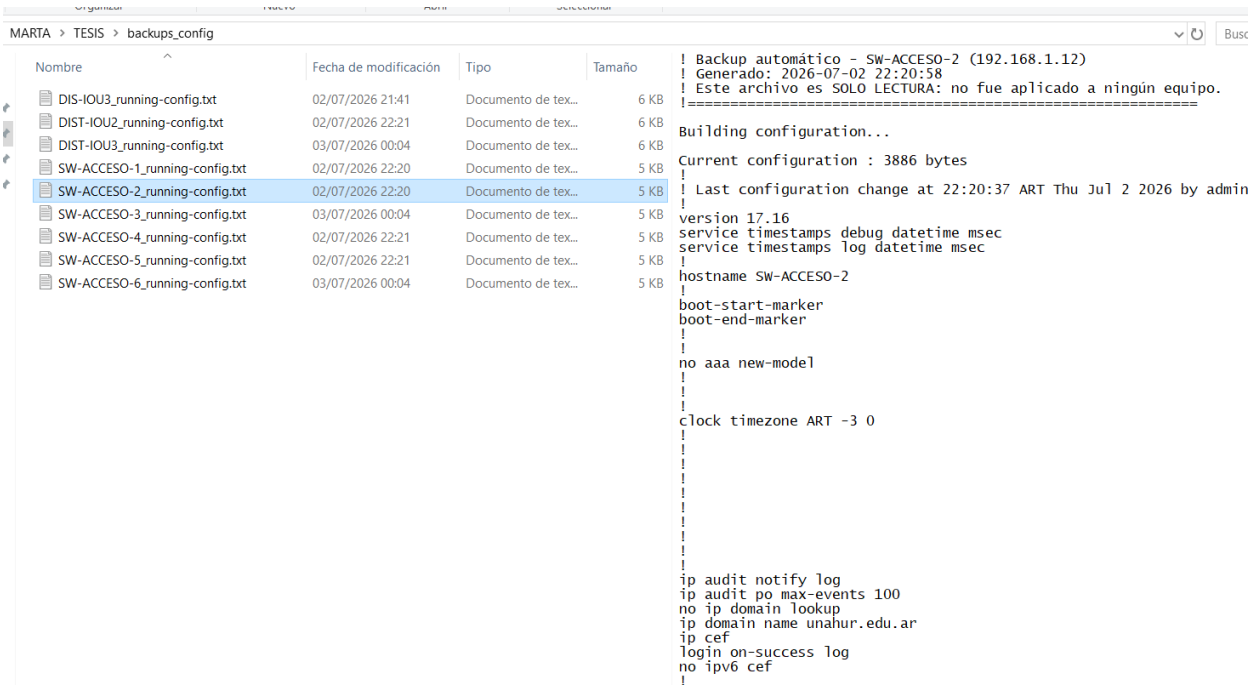
Nº	Acción	Capítulo	Dificultad	Tiempo estimado
1	Definir problemática y objetivos	Cap. 1	🟢 Baja	1 semana
2	Diseñar topología jerárquica	Cap. 2	🟡 Media	1 semana
3	Configurar GNS3 VM + Cloud	Cap. 2	🟡 Media	1 semana
4	Aprovisionar imágenes Cisco IOL	Cap. 3	🟡 Media	1 semana
5	Plan de direccionamiento + DHCP Relay	Cap. 3	🔴 Alta	2 semanas
6	Construir archivo YAML	Cap. 4	🟢 Baja	1 semana
7	Desarrollar script Python con Netmiko	Cap. 4	🔴 Alta	2 semanas
8	Implementar idempotencia y compliance	Cap. 4	🟡 Media	1 semana
9	Configurar troncales dot1q	Cap. 5	🟡 Media	1 semana
10	Blindar accesos SSHv2	Cap. 5	🟡 Media	1 semana
11	Silenciar syslogs de consola	Cap. 5	🟢 Baja	0,5 semana
12	Verificación de enlaces y tablas MAC	Cap. 6	🟡 Media	1 semana
13	Auditoría final de puertos y pruebas	Cap. 6	🔴 Alta	1 semana

Anexo D – Evidencia de Auditoría y Verificación de Cumplimiento

El motor de automatización desarrollado no solo demuestra idempotencia y compliance sobre el plano de configuración (VLANs y puertos), sino que fue sometido a una auditoría final de verificación sobre las capas de resiliencia de la red — Spanning-Tree, alta disponibilidad del gateway (HSRP) y hardening de acceso remoto — con el objetivo de constatar, con evidencia documental extraída directamente del hardware emulado, que la totalidad de las directivas de diseño descriptas en los Capítulos 3 y 5 se encuentran efectivamente implementadas y operativas en la maqueta final.

D.1 Verificación de Alcance del Motor de Backup

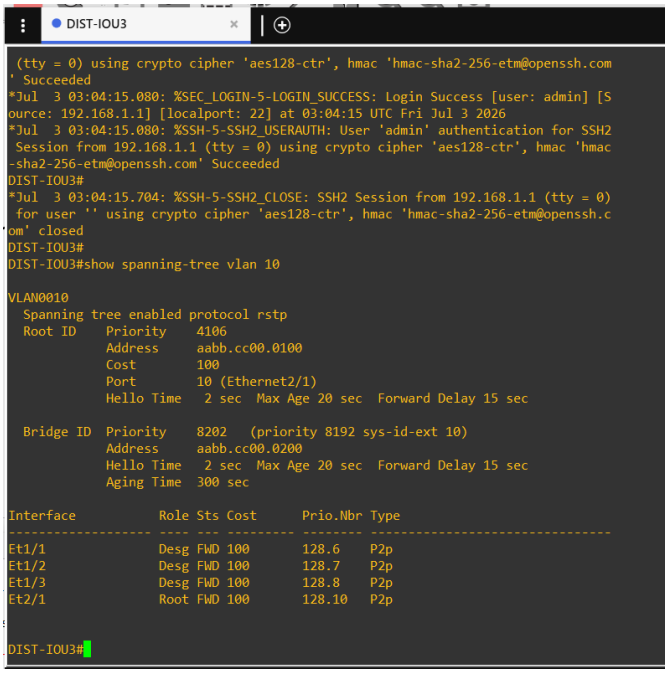
Como mecanismo complementario de resguardo, se incorporó al motor de automatización la función backup_running_config(), encargada de ejecutar el comando de solo lectura show running-config sobre cada dispositivo de la topología y persistir su resultado en un archivo de texto individual, sin enviar en ningún caso comandos de configuración al equipo. Esta función opera como una capa de auditoría fuera de banda respecto del ciclo principal de compliance: mientras que aplicar_compliance_switch() verifica y corrige desvíos activos, backup_running_config() documenta el estado final resultante, generando así la trazabilidad necesaria para reconstruir la configuración de cualquier equipo ante una eventual pérdida de NVRAM, sin depender de la memoria del operador ni de capturas de pantalla manuales.



D.2 Evidencia de Convergencia de Spanning-Tree (Rapid-PVST+)

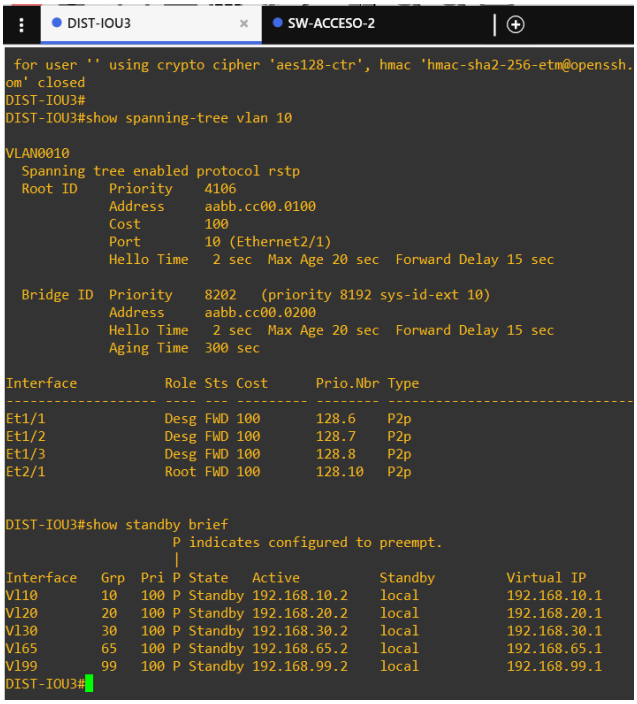
Con el objetivo de validar empíricamente la mitigación de bucles de Capa 2 descripta en la Sección 5.4, se ejecutó el comando de verificación show spanning-tree vlan 10 sobre ambos nodos de distribución. La salida obtenida confirma que DIST-IOU2 fue electo Root Bridge (prioridad 4106, incluyendo el offset de sys-id-ext), mientras que DIST-IOU3 se reconoce a sí mismo como nodo no-raíz (prioridad 8202) y marca su enlace troncal hacia DIST-IOU2 (interfaz Et2/1) en estado Root FWD. Esta evidencia confirma que la topología converge de manera

determinista según el diseño declarativo establecido, sin depender del desempate por dirección MAC.



D.3 Evidencia de Alta Disponibilidad HSRP

Para verificar la correcta implemen
tación del mecanismo de alta disponibilidad del default-gateway descripto en la Sección 5.5, se ejecutó el comando show standby brief sobre DIST-IOU2. La salida confirma que dicho nodo opera en estado *Active* para la totalidad de las VLANs de producción (10, 20, 30, 65 y 99), con DIST-IOU3 registrado como *Standby* en cada grupo y la correspondiente dirección IP virtual compartida por ambos nodos. Esta verificación certifica que, ante la falla del nodo de distribución primario, el tráfico de los hosts finales continuaría siendo enrutado sin interrupción perceptible a través del nodo secundario.



D.4 Matriz Final de Cumplimiento de Hardening VTY

Como cierre del proceso de auditoría, se relevó el estado del transporte habilitado en las líneas VTY de la totalidad de los ocho dispositivos gestionados, con el fin de confirmar el cumplimiento homogéneo de la directiva de hardening establecida en la Sección 5.2 (uso exclusivo de SSH Versión 2, sin excepciones).

Equipo	`transport input` configurado	Estado
SW-ACCESO-1	ssh	✅ Conforme
SW-ACCESO-2	ssh	✅ Conforme
SW-ACCESO-3	ssh	✅ Conforme
SW-ACCESO-4	ssh	✅ Conforme
SW-ACCESO-5	ssh	✅ Conforme
SW-ACCESO-6	ssh	✅ Conforme
DIST-IOU2	ssh	✅ Conforme
DIST-IOU3	ssh	✅ Conforme

La totalidad de los equipos de la topología rechaza actualmente conexiones no cifradas vía Telnet, cerrando así la superficie de exposición que había quedado abierta durante la fase preliminar de configuración manual (Anexo A).

Nota sobre la demostración en vivo

La totalidad de los hallazgos documentados en este anexo puede ser reproducida en vivo durante la defensa, ejecutando el motor de automatización (Script_de_Automatización_NetDevOps.py) contra la maqueta activa en GNS3, y contrastando en tiempo real la salida de consola con el contenido de los archivos reporte_completo_automatizacion.txt y backups_config/ generados automáticamente.

Anexo E – Cuestionario de Validación, Escalabilidad y Conceptos Clave (FAQ)

Eje 1: Arquitectura, Crecimiento y Escalabilidad

1. ¿Qué significa que el proyecto sea "escalable" y cómo se logra?

Explicación: Que un sistema sea escalable significa que tiene la capacidad de crecer y manejar un volumen mucho mayor de trabajo sin perder eficiencia y sin necesidad de rediseñar su estructura desde cero. **En este proyecto:** Se logra gracias a dos factores:

- *Estructura Jerárquica (Física):* La red está organizada como una empresa bien estructurada (un núcleo central o "Core", áreas de distribución y terminales de acceso). Se pueden añadir más oficinas o switches de acceso en los extremos sin alterar el funcionamiento del núcleo.
- *Automatización por Código (Lógica):* Toda la información de la red está escrita en un único archivo de texto (config_red.yaml) que funciona como los planos maestros de un edificio. Si la red crece de 6 a 60 dispositivos, no hace falta reescribir el programa de Python; solo se añaden los datos de los nuevos equipos en ese plano de texto y el sistema los configura automáticamente.

2. ¿Cuáles son los límites que condicionan el crecimiento de esta red?

Explicación: Los límites no están en el diseño del programa, sino en las restricciones de los recursos materiales y matemáticos disponibles:

- *Límites de hardware:* Al ser un entorno de simulación para pruebas, el crecimiento está topado por la memoria RAM y el procesador de la computadora que ejecuta el laboratorio virtual.
- *Límites matemáticos de direccionamiento (IP):* Las direcciones de red se agrupan en paquetes lógicos llamados subredes. En la configuración actual, cada una de las 5 áreas lógicas (VLANs) permite un máximo matemático de 254 computadoras conectadas en simultáneo, dando un límite elástico de aproximadamente **1.270 dispositivos**. Si la organización supera esa cantidad, simplemente se debe modificar la máscara matemática de la red para ampliar el cupo.

3. ¿Qué otros componentes tecnológicos se podrían sumar a este sistema?

Explicación: El programa de automatización actúa como un "director de orquesta" que puede leer partituras de diferentes instrumentos. No está limitado a switches; se pueden integrar dispositivos de seguridad (Firewalls/Cortafuegos), enrutadores de internet de gran escala o sistemas de conexión inalámbrica (antenas Wi-Fi corporativas). El único requisito es que el dispositivo permita una vía de comunicación de texto estándar (SSH).

Eje 2: Filosofía del Código y Operaciones Avanzadas

4.¿Qué es la "Idempotencia" y por qué es vital en este proyecto?

Explicación: La idempotencia es una propiedad lógico-matemática donde el resultado de una operación es siempre el mismo, sin importar cuántas veces se ejecute.

- *Analogía:* Pensémoslo como un interruptor de luz inteligente con la orden "Encender". Si la luz está apagada, la enciende. Si la luz ya estaba encendida, no hace nada; no genera un cortocircuito ni titila.
- *El caso real del proyecto:* Al desplegar la nueva área de *Recursos Humanos (VLAN 65)*, el script detectó que los switches del 1 al 3 ya la tenían configurada, por lo que los ignoró para no interrumpir su transmisión actual. En cambio, en los switches del 4 al 6 (que no la tenían) inyectó la configuración en solo **14 segundos, logrado en cada una de las 20 repeticiones**. Esto demuestra que el sistema es

seguro y predecible: estabiliza la red sin provocar caídas innecesarias ni requerir cortes de servicio programados fuera del horario laboral.

5. ¿Por qué se afirma que el script funciona como un "Auditor de Cumplimiento" (Compliance) y no como un simple tirador de comandos?

Explicación: Un "tirador de comandos" tradicional ejecuta órdenes a ciegas, machacando la red sin importar el estado actual (lo que puede causar errores o sobrescribir datos correctos). Este script actúa con **inteligencia de cumplimiento**: primero interroga al equipo, analiza matemáticamente su estado actual y lo compara con el plano maestro (config_red.yaml). Si el estado del equipo coincide con el plano, el script finaliza su tarea de inmediato **sin consumir recursos de procesamiento (CPU)** del hardware. Solo interviene de manera quirúrgica si detecta un desvío o una falla de seguridad.

Eje 3: Seguridad y Gestión de Datos

6. ¿Cómo controla el script los riesgos de seguridad en los puertos de conexión física?

Explicación: Un peligro común en seguridad informática es que alguien conecte una computadora no autorizada en una toma de red vacía en una oficina. El script mitiga esto mediante una política estricta basada en el plano maestro:

- Revisa el estado físico de los conectores. Si detecta que un puerto que fue catalogado como inactivo en el plano maestro se encuentra encendido (porque alguien se conectó físicamente sin permiso), el algoritmo interpreta esto como una anomalía crítica e inyecta inmediatamente la orden de apagado lógico (shutdown). Así, bloquea de raíz cualquier intento de intrusión física o conexión de hardware malicioso.

7. Además de mostrar textos en la pantalla, ¿dónde se guardan los resultados de las pruebas? ¿El script guarda automáticamente los resultados de las pruebas en un archivo o bitácora en este momento?

Explicación: No, en este momento el script no lo hace de forma automática; actualmente muestra los resultados en tiempo real directamente en la pantalla de la terminal para que el operador los lea. Pero puedo hacerlo. Para que funcione de forma autónoma, se pueden programar dos mejoras con herramientas estándar

- Automatización del tiempo (Cron/Task Scheduler): Se puede configurar el sistema operativo (mediante un comando o tarea programada) para que ejecute el script de Python de forma automática dos veces al día (por ejemplo, a las 8:00 y a las 20:00 hs), sin que un humano tenga que apretar "Play".
- Módulo de Alertas (Telegram Bot): Se puede añadir una función en Python para que, en caso de detectar una anomalía (como un puerto encendido sin permiso), el script no solo lo registre en un archivo de texto en la computadora, sino que realice una llamada web instantánea a un bot de Telegram. Este bot enviaría un mensaje inmediato al celular del equipo de IT detallando: la anomalía registrada,

el día, la hora exacta y la subsanación (la acción correctora que aplicó el script para solucionar el problema).

Eje 4: Limitaciones y Futuro del Proyecto

8. ¿Qué limitaciones tiene el sistema actual y cómo podría evolucionar?

Explicación: Actualmente, el sistema requiere que el operador tenga conocimientos técnicos mínimos para ejecutar el código desde una consola de comandos y simula la escritura humana sobre los equipos. Para transformar este proyecto en un producto comercial o de uso diario para técnicos principiantes, el mapa de ruta contempla:

- *Interfaz Visual:* Diseñar una página web interna con botones y gráficos sencillos para que no requiera usar líneas de comandos.
- *Alertas en tiempo real:* Integrar el envío de mensajes automáticos a canales institucionales (como un bot de Telegram o Slack) cada vez que el auditor automático repare una falla.
- *Programabilidad pura:* Migrar la comunicación hacia formatos modernos de intercambio de datos (como APIs con modelos YANG), lo que permitiría que el script hable directamente el idioma nativo de las computadoras de red, acelerando la velocidad de procesamiento de minutos a milisegundos.

Nota Técnica: Persistencia de NVRAM en Imágenes Cisco IOL y Prontuario para Demostración en Vivo

Dato de Color: Volatilidad de la Memoria NVRAM en Cisco IOL

Es importante documentar un comportamiento recurrente observado durante las sesiones de laboratorio, para diferenciarlo claramente de un error de diseño en el motor de automatización. Las imágenes Cisco IOL (IOS on Unix), al tratarse de binarios nativos ejecutados sobre un entorno Linux virtualizado (GNS3 VM) en lugar de emular hardware físico completo, gestionan su memoria NVRAM mediante un mapeo a archivos del sistema de archivos del host, en vez de un chip de almacenamiento persistente dedicado como en un equipo físico real.

Como consecuencia de esta arquitectura, se observó en reiteradas ocasiones que, pese a la correcta ejecución del comando write memory (confirmado por la salida %SYS-5-CONFIG_I: Configured from console by console), las claves criptográficas RSA generadas para el servicio SSH podían perder su estado activo tras interrupciones del proceso IOU (por ejemplo, ante alta carga de CPU del host, suspensión parcial del hipervisor, o reinicio del proceso GNS3 VM), quedando el servicio SSH momentáneamente deshabilitado (ip ssh en estado DISABLE) aunque el resto de la configuración —VLANs, puertos, banners— permanecía intacto.

Este comportamiento es una limitación conocida y documentada del entorno de emulación Cisco IOL sobre GNS3, ajena a la lógica de los comandos configurados y a la calidad de la persistencia gestionada por el motor de automatización desarrollado. Ante su ocurrencia, la solución aplicada es siempre la misma y no altera ningún otro parámetro del equipo:

```
configure terminal
crypto key generate rsa general-keys modulus 2048
end
write memory
```

Prontuario de Comandos para Demostración en Vivo

Con el objetivo de que el tribunal evaluador pueda presenciar en tiempo real el funcionamiento del principio de Idempotencia (Capítulo 4.3), se documentan a continuación los comandos manuales necesarios para revertir de forma controlada el estado de un switch de acceso a una condición de "desviación" previa a la ejecución del script — sin alterar en ningún momento los puertos principales de producción (troncales activos ni puertos de acceso ya asignados a sus VLANs correspondientes).

Paso 1 — Eliminar una VLAN de prueba (simular desviación de compliance):

```
configure terminal
no vlan 65
end
write memory
```

Verificación: show vlan brief debe confirmar que la VLAN 65 ya no aparece en la tabla.

Paso 2 — Simular un puerto residual encendido sin autorización (para demostrar el hardening de seguridad):

configure terminal
interface Ethernet 0/3
no shutdown
end
Nota: la interfaz Ethernet 0/3 fue seleccionada por estar catalogada como inactiva en config_red.yaml para este switch — es segura de manipular con fines demostrativos porque no participa del tráfico de producción (no es un puerto troncal ni un puerto de acceso asignado a una VLAN de usuario).
Paso 3 — Ejecutar el motor de automatización y observar en vivo:
Al correr Script_de_Automatización_NetDevOps.py frente al tribunal, el motor deberá:

Detectar [DESVIACIÓN DETECTADA] para la VLAN 65 faltante y recrearla automáticamente.
Detectar que Ethernet 0/3 está activa cuando debería estar apagada, e inyectar el shutdown correctivo.
Informar [COMPLIANCE OK] en el resto de las interfaces, sin tocarlas.

Paso 4 — Verificación final:
show vlan brief
show interfaces status
Ambas salidas deben coincidir nuevamente con el estado declarado en config_red.yaml, cerrando el ciclo demostrativo de auditoría → desviación → corrección → cumplimiento, en presencia del tribunal.Dato de color, GNS3, ante cualquier corte o quedar la maquina en suspensión, al estar en el entorno de Windows, puede apagar los SSH de los switch. Ante cualquier error verificar
Show ip ssh
En caso de dar DISABLE, encenderlo:
crypto key generate rsa general-keys modulus 2048

Borrar

conf ter

no vlan 65

no vlan 30

interface Ethernet 0/3

no shutdown

end

write memory

show vlan brief

Abrir - up

configure terminal

interface Ethernet 0/3

no shutdown

show interfaces status.